A project report on

Development of Deep Lerning Techniques for Depression Detection

Submitted in partial fulfilment for the award of the degree of

Bachelor of Technology in Electronics and Computer Engineering

by

AYUSH NEGI (22BLC1259)

AYUSH ARORA (22BLC1218)

KARAN GHUWALEWALA (22BLC1201)

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF ELECTRONICS ENGINEERING

November,2025

VIT

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Declaration

I hereby declare that the report titled *"Development of Deep Learning Techniques for Depression Detetion"* submitted by me to the School of Electronics Engineering, Vellore Institute of Technology, Chennai in partial fulfillment of the requirements for the award of Bachelor of Technology in Electronics and Computer Engineering is a bona-fide record of the work carried out by me under the supervision of *Prof. Ashok* .

I further declare that the work reported in this report, has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or University.

Sign:

Name & Reg. No.: Karan Ghuwalewala & 22BLC1201

Date: 12/11/25

VIT

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Electronics Engineering

Certificate

This is to certify that the project report titled *Plant stress detection for sustainable agriculture* submitted by *Karan Ghuwalewala* (22BLC1201.) to Vellore Institute of Technology Chennai, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Electronics and Computer Engineering is a bona-fide work carried out under my supervision. The project report fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature: .....

Name: Dr. Ashok Mondal

Date: 11/11/25

Supervisor

Signature: .....

Name: Dr. Vijay Kumar P

Date: 12/11/25

Head of the Department

Head of the Department
B.Tech.(ECM) Programme
School of Electronics Engineering
Vellore Institute of Technology(VIT)
Vandalur - Kelambakkam Road,
Chennai - 600 127.

Examiner

Signature: .....

Name: Dr. KIRUTHIKA. V

Date: 12/11/25

(Seal of the School)

Vellore Institute of Technology
School of Electronics Engineering
Chennai - 600127, Tamil Nadu, India

Abstract

Depression affects millions of people around the world, but identifying it early is still a challenge. Most diagnosis methods depend on face-to-face interviews, written questionnaires, and the patient's willingness to express their feelings honestly. However, many individuals either avoid talking about their emotional state or are unaware of their symptoms. This delay in detection often leads to worsening mental health conditions.

In this project, we explore a different idea — *can depression be detected through the way a person speaks and what they speak.* Research shows that people dealing with depression often have noticeable changes in their voice, such as low energy, lack of pitch variation, slower speech, and frequent pauses. We aim to capture these subtle speech patterns as well as patterns in text showing depressive thinking using Deep Learning.

We will be using the DAIC-WOZ clinical dataset, which contains real interview recordings which are designed for depression analysis. The text transcripts are converted into question answer pairs using interrogative words along with their labels. The audio recordings are first cleaned by removing background noise and normalizing. We then convert each recording into a Mel-spectrogram which is a visual form of sound that shows frequency, pitch, and energy change over time. The question answer pairs then passed unto the BERT tokenizer while the spectrogram images are used as input to the Vision Transformer (ViT) model.

The tokenizer converts the text into vector embeddings which is then passed through 12 encoders which give final 786 dimension vector which is like the summary of the Q&A pair. Finally, a classification layer predicts whether the speaker is "Depressed" or "Not Depressed." The Vision Transformer treats each spectrogram like an image: it divides the image into small patches, analyses relationships between them using self-attention, and learns emotional speech characteristics. Finally, a classification layer predicts whether the speaker is "Depressed" or "Not Depressed."

After hyperparameter tuning, the ViT model improved from 55% to 69% accuracy on audio-based depression detection and the BERT models saw an improvement from 64% to 76%. These results show that speech and text contain meaningful indicators that transformer-based models can recognize and classify.

This project demonstrates that an AI-based approach can support early screening of depression — without needing the person to talk about their emotions directly — making mental health assessment more accessible, especially for remote or telehealth applications.

Acknowledgements

We wish to express our sincere thanks and deep sense of gratitude to our project guide, Dr. Ashok, Associate Professor, School of Electronics Engineering, for his consistent encouragement and valuable guidance offered to us in a pleasant manner throughout the course of the project work.

We are extremely grateful to Dean Dr. Ravi Sankar A, Associate Dean (Academics) Dr. Mohana Prasad K, & Associate Dean (Research) Dr. Ashok, of the School of Electronics Engineering, VIT Chennai, for extending the facilities of the school towards our project and for his unstinting support.

We express our thanks to our Head of the Department Dr. Vijayakumar P for his support throughout the course of this project.

We also take this opportunity to thank all the faculty of the School for their support and their wisdom imparted to us throughout the course.

We thank our parents, family, and friends for bearing with us throughout the course of our project and for the opportunity they provided us in undergoing this course in such a prestigious institution.

TABLE OF CONTENTS

Declaration i

Certificate ii

Acknowledgement iii

Abstract iv

Chapter 1 – Introduction 9

1.1 Background of the Study

1.2 Problem Statement

1.3 Motivation

1.4 Objectives

1.5 Scope of the Project

1.6 Major Contributions

Chapter 2 – Literature Survey 13

2.1 Text-Based Depression Detection Models

2.2 Audio-Based Depression Detection Models

2.3 Multimodal Depression Detection Models

Chapter 3 – Methodology 16

3.1 Dataset – DAIC-WOZ

3.2 Experimental Setup

3.2.1 Overall Setup

3.2.2 Tools and libraries used for text

3.2.3 Tools and libraries used for audio

3.3 Text analysis

3.3.1 Data Preprocessing

3.3.2 BERT Model Architecture

3.4 Audio Architecture

3.4.1 Data Processing Pipeline

3.4.1.1 Audio → Mel-Spectrogram Conversion

3.4.1.2 Class Imbalance Handling

3.4.2 Workflow Overview

3.4.2.1 Visualizing Audio Signals

3.4.2.2 Power Spectrum Analysis

3.4.2.3 Generating Mel-Spectrograms

3.4.3 Organizing Spectrograms into Classes

3.4.3.1 Automation Script

3.4.4 Data Handling

3.4.4.1 Google Drive Storage Setup

3.5 Vision Transformer (Model)

3.4.5.1 Preparing Data for ViT

3.4.5.2 ViT Architecture (simplified)

3.4.5.3 How ViT Works

3.4.6 MLP (Final Layer)

3.4.7 Data Format

Chapter 4 – Results & Discussion 30

4.1 Text-Based Model (BERT)

4.2 Audio-Based Model (ViT)

4.3 Comparison with Previous Work

4.3.1 Text Modality

4.3.2 Audio Modality

4.4 Discussion

Chapter 5 – Conclusion & Future Scope 36

5.1 Conclusion

5.2 Future Scope

Appendix 38

A. Pseudocode – Text Pipeline (BERT)

B. Pseudocode – Audio Pipeline (ViT)

References 60

Chapter 1 – Introduction

1.1 Background of the Study

Depression is becoming one of the largest mental health crises in the word. According to research conducted by the Word Health Organization (WHO), more than 332 million people across the globe are struggling from depression of some kind. Depression is currently classified as one of the leading causes of disability by some sources which goes to show how serious of an issue it is becoming.

Even though depression is on a steady rise, current methods of diagnosing depression consist of interviews, counselling sessions, or self-report questionnaire. These are good methods but are held back by the fact that if the patient is not willing to say or reveal everything correctly. Mental health conditions like depression are not something that we can do some kind of physical test for and find out if someone has it or not. As many people do not fully talk about or revel their inner thoughts and feelings due to fear, social stigma, or lack of awareness. Due to this depression goes undiagnosed in many people. If depression remains untreated it affects a person's emotional and physical health as well. It has an effect on every area of a person's life ranging from worsening of interpersonal relations ships, not being able to focus on one's career. It leads to increase risk of suicide as well as physical complications like weight gain or obesity further leading to heart conditions and diabetes.

Previous research in this field has shown that depression does not affect just the emotions of person but also their way of speak and what kind of words they end up using. Affected individuals often exhibit:

- Low speech energy (low tone or soft voice)
- Monotone or flat pitch (voice sounds emotionless)
- Long pauses and hesitation while speaking
- Higher use of negative emotional words

Because these patterns exist in voice and language, AI can help detect the signs of depression without really asking someone any personal questions or forcing people to describe how they feel.

1.2 Problem Statement

Early identification of depression is difficult due to:

- There is a shortage of mental health professionals across the globe. This is especially true in developing and rural areas. This all leads to many people having the facility of being diagnosed by a mental health professional on time
- In our society there is a social stigma around having mental health issues this causes people to have shame and keep their emotions pent up and not opening up to others even if they are mental health professionals trying to help them.
- There is a lack of awareness among many people about the effects and symptoms of mental health issues which lead to people not understanding the severity of their conditions
- Currently existing traditional methods rely on the ability of the patient to self-disclose or the interviewer's ability to get the patient to open up. This can be a difficult task and require multiple appointments which cause delays and some people may not be able to afford such long treatments.
- These barriers lead to many people being misdiagnosed or delayed intervention

So, there is a strong need for an automated, objective, and non-intrusive system that can detect depression without the need for manual evaluation.

1.3 Motivation

Our motivation to work on this project is:

- Developing an AI system for detection which can reach a large number of people especially those in developing or rural places.
- Early detection of depression can prevent the condition from becoming worse and reduce the risk of complications
- Speech and language are natural forms of communication this means that samples for them can be easily collected using things like mobiles apps without having to ask patients intrusive questions which they may not want to answer.

- Modern transformer models like BERT and Vision Transformer learn patterns much better than older machine learning approaches.

If depression changes how we speak, then deep learning can learn those changes.

1.4 Objectives

Our goals with this project are:

- Convert the text transcripts to question and answer pairs and tokenize them.
- Train a BERT to classify these tokens into depressed or not depressed.
- Convert the audio recordings of the interviews into Mel-spectrogram images.
- Train a Vision Transformer (ViT) to classify them as depressed or not depressed.
- Compare text-based detection vs. audio-based detection.

1.5 Scope of the Project

- Dataset Used: DAIC-WOZ (Distress Analysis Interview Corpus – Wizard of Oz) it consists of recording of interviews done between a patient and virtual assistant "Ellie", text transcripts of the recorded interviews and the labels taken from clinical diagnosis
- Modalities used by us:
    - Text (conversation transcripts) – Classified using BERT
    - Audio (interview recordings)
- Final Output obtained:
    - Patient is Depressed (1)
    - Patient is Not-Depressed (0)
- Limitations: Our project does not include video features or facial expressions.

Our project focuses only on classifying depression in the patient based on thier speech and language patterns.

1.6 Major Contribution

Component	Contribution
Text Model (BERT)	Achieved an accuracy of 76%. The model demonstrates strong ability to detect depressive linguistic patterns.
Audio Model (ViT)	Achieved an accuracy of 69%. The model captures vocal features like monotone pitch, reduced loudness, longer pauses associated with depression
Findings	Both voice and language are good indicators of depression in a person and deep learning models can identify these signs with promising accuracy

Chapter 2 – Literature Survey

2.1 Text-Based Depression Detection Models

Zang et al [1] gave a model which used multi-instanced learning framework using multi-task transformer model (multi-MT5 and multi-RoBERTa). In their work they segmented long multi round dialogue and extracted depression related features from them. They achieved an F1 score of 0.88. They also made use of LIME which was used to tell which sentences exactly were the most important for classification, this enhanced their clinical explainability because their model was not like a black box which one couldn't figure out why it had predicted someone as depressed or not.

Milintsevich et al [2] used a hierarchical regression model which was based on RoBERTa. They instead of predicting a binary label for depression focused on predicting the individual PHQ-8 scores of an individual based on the text transcripts. When their model was evaluated on binary classification it achieved a F1-score of 0.739.

Burdisso et al [3] used only the participant responses from the DIAC-WOZ transcripts and ignored the Ellie lines. They did this as they believed that the interviewer Ellie only asked revealing questions to people it considered to be suffering from depression. Hence taking into account her responses would lessen the reliability of the model in real world scenarios. They used a RoBERTa model and trained it only on the participant responses and achieved an F1-score of 0.75

Teferra et al [4] explored the use of modern large language models (LLMs) like GPT-4o, Llama 3 and Gemini. They did this is a zero-shot setting which means that they did this without fine tuning these models. They gave these models the transcripts and used sophisticated prompts to predict the PHQ-8 scores of the participants. Their model achieved a the max F1-score of 0.74 using GPT-4o.

Tank et al [5] fine-tuned a RoBERTa and DepRoBERTa model on the interview transcripts of the DIAC-WOZ dataset. Their fine-tuned model achieved a classification accuracy of 71.43% providing a solid baseline performance for a transformer-based model for text prediction.

2.2 Audio-Based Depression Detection Models

Triohin et al [6] investigated on how to convert a Convolutional neural network (CNN) which had been trained on audio spectrograms of the interview present in

DAIC-WOZ dataset into a more efficient Spiking neural network (SNN). Their base CNN model achieved an F1-score of 0.825. After successfully converting this CNN model to SNN model, their SNN model matched the performance of their CNN model. Their study showed that there is a path towards low-power and real time audio screening.

Lin et al [7] created an end-to-end audio model. They created a model combining 1D CNN and BiLSTM. This model took raw audio as input which had been segmented into 15 second chunks. The 1D CNN learnt features from the waveform while the BiLSTM was to model temporal sequences. Their model achieved an precision of 0.77, showing that even without pre-defined feature sets models can learn biomarkers.

Zhou et al [8] created a CNN model. They extracted all kinds of features such as low-level descriptors (LLDs), Mel-frequency cepstral coefficients and functionals. These features after being extracted from the audio are passed into the CNN. Using these features their CNN model achieved an accuracy of 0.83.

Ravi et al [9] used an audio-based deep learning architecture called 'DepAudioNet'. For this model they extracted a set of 6373 static features (LDDs and their functionals) using the openSMILE tools. This specialized CNN based model that they created was designed in such a way that it effectively processed the large feature set of theirs. With this model they achieved an accuracy of 0.74.

Feng et al [10] extracted handcrafted features from the audio interviews present in the DAIC-WOZ dataset. To this set of acoustic features, they passed through a CNN model to classify them. Using this their model achieved an accuracy of 0.77, which reinforce the fact that standard acoustic features are highly effective for this task.

2.3 Multimodal Depression Detection Models

Nykoniuk et al [11] compared early and late fusion models as well as unimodal models. They used CNN for audio classification and BiLSTM for text classification. Their late fusion model took a weighted average of the prediction of each unimodal modal while the early fusion model concatenated the features which were extracted from both audio and text and used this concatenated feature set to classify the participants as depressed or not depressed. Out of the four models, their early fusion model performed the best having the highest F1-score of 0.79.

Iyortsuun et al [12] propose a 'Additive Cross-Modal Attention (ACMA) network. This model BiLSTM as the backbone for text and audio classification but on top of

that an attention mechanism has been added. This attention mechanism allows the model to learn what the interactions between the two modalities are and to weigh their importance dynamically. Their model achieved an F1-score of 0.82.

Jo and Kwak [13] built a four-stream model. They had two streams for audio which were MFCCs and spectrograms and two streams for words which were word embeddings and n-grams. For each representation of the data, they trained a sperate deep learning model. The final classification was done by taking ensemble of SoftMax output of all four streams.

Aghaei and Khodaei [14] propose a 'novel attention fusion network'. They integrate the features from text, audio as well as video. They focus on finding the best way to combine these three modalities using an attention mechanism such that it weighs its features dynamically.

Yang et al [15] propose a model which uses Gate Recurrent Unit (GRU) for audio classification and BiLSTM with attention layer for text classification. The GRU will work on acoustic features for audio. The features from both these models will then be combined in a early fusion architecture and a classifier will be used to further classify these features.

Chapter 3 – Methodology

3.1 Dataset – DAIC-WOZ (Distress Analysis Interview Corpus – Wizard of Oz)

We have taken the DAIC-WOZ (Distress Analysis Interview Corpus – Wizard of Oz) dataset. Distress Analysis Interview Corpus means that it is a collection (corpus) of interviews, taken to analyse the signs of distress. Wizard of Oz refers to the fact the interview is being taken by the virtual agent "Ellie" who in reality is being controlled by a human behind the scene similar to the wizard behind the curtain in the Wizard of Oz. It a set of interviews in which in which participants interact with a virtual agent named "Ellie" who is being controlled by a human. It is a multimodal dataset and contains:

- Audio recordings of the interview
- Text transcripts of the recorded interview
- Video features extracted from the recordings of the participants

For our project we have used the audio and text data. The audio data is present in wav format which is the raw recordings of the interview. The text data is in csv format and it contains the columns start time, stop time, speaker, and value. The start and stop time tell when a speaker is speaking, the speaker column has two values "Ellie" and "Participant" to tell which one is speaking and value is the text that they have spoken. The participants are assigned labels are depressed or not depressed based on their PHQ-8 score. PHQ-8 (Patient Health Questionnaire-8) is a standard clinical Questionnaire which is used to diagnose depression. It consists of 8 questions which patients can give answers which are then given 0,1,2 or 3 points. A score of 10 or higher is used as a sign of depression. Any person who has a score of more than 10 is given the label depressed and rest are given not depressed in the dataset. The dataset contains 189 such interview.

3.2 Experimental Setup

3.2.1 Overall Setup

For this project as we needed to train large models our own systems proved to be insufficient. To remedy this we have made use of the free online resource called Colab which is being provided by google. It can be used in a way similar to jupyter notebooks. We have used it to run our code using the T4 GPU hardware accelerator which provides us with 12.7 GB of System RAM and 15 GB of GPU RAM. For import our dataset we have first uploaded it to our google drive and then used the mount

feature in Colab to mount our drive to the Colab notebook. When the drive is mounted all of the files present in it can be easily accessed by

```
from google.colab import drive

drive.mount('/content/drive')
```

This allows us to read/write data easily.

3.2.2 Tools and libraries used for text

Tool / Library	Purpose
Python	Programming
Pandas	Handling our dataset
NumPy	Numerical operations
Transformers	BERT model
Torch	To create tensors
Sklearn	evaluation

3.2.3 Tools and libraries used for audio

Tool / Library	Purpose
Python	Programming
Librosa	Load audio + create spectrograms
NumPy	Numerical operations
Matplotlib	Visualizing signals and spectrograms
PyTorch / TensorFlow	Vision Transformer model

3.3 Text analysis

3.3.1 Data Preprocessing

For text there are 189 transcripts each of which contain the columns start time, stop time, speaker, and value.

Start time	Stop time	Speaker	Value
60.028	61.378	Ellie	how are you doing today
62.328	63.178	Participant	good
63.798	64.738	Ellie	that's good
65.858	67.528	Ellie	where are you from originally
68.978	70.288	Participant	atlanta georgia

Table 1: DAIC WOZ Text Transcript

For our project we extracted question answer pair from the transcripts. For this we have used a list of interrogative words:

INTERROGATIVE_WORDS = ("what", "when", "where", "which", "who", "whom", "whose", "why", "how", "is", "are", "am", "was", "were", "do", "does", "did", "has", "have", "had", "can", "could", "will", "would", "should", "may", "might", "tell me", "describe", "explain", "give me an example", "so,", "and what", "what about", "how about")

Any sentence by Ellie beginning with these words is taken as a question and the participant's lines till the time Ellie next speaks is taken as the answer to that question. The pairs containing empty lines are also removed. After this the PHQ_Binary assigned to the participant are added to each pair. The dataset now looks like Participant_ID, question, answer, and PHQ_Binary.

Participant_ID	Question	Answer	PHQ_Binary
300	how are you doing today	good	0
300	where are you from originally	atlanta 19eorgia	0
300	why'd you move to l_a	um my parents are from here um	0
300	how do you like l_a	i love it	0
300	what are some things you really like about l_a	i like the weather i like the opportunities um yes	0

Table 2: Question Answer pair with labels

The dataset obtained by this is imbalanced it contains 2133 Not Depressed samples and 1442 Depressed samples. To fix this imbalance we have applied Random oversampling on the minority class (Depressed). This randomly picks a sample in the minority class and duplicates it till both samples are the same (2133 samples).

Participant_ID	Question	Answer	PHQ_Binary
325	are you okay with this	yes	1
357	how are you at controlling your temper	pretty good i don't get angry very often	0
304	how close are you to them	um very close we talk every day	0
320	what's the hardest thing about being a parent	um i think decision making	1

312	have you been diagnosed with depression	nope	0

Table 3: Final balanced data after random oversampling

3.3.2 BERT Model Architecture

Bidirectional Encoder Representations from Transformers (BERT) is a language model developed by Google. It is pretrained on a large amount of text which comes from Wikipedia articles and BookCorpus dataset. It takes sentences from both directions so it can understand the context of the sentence. Our input Q&A pairs are first passed through the BERT tokenizer. It first combines our Q and A into a single string where they are separated by a [SEP] token. It then adds a [CLS] token to the beginning and [PAD] tokens to the end to pad the sequence to 128 tokens long. We have taken the max length of the sequence that is 128. The tokens are then converted to their Numerical Input_IDs these point to BERT's internal table which contain 768-dimensional vector for each token where similar meaning words have similar vectors. Also, an attention mask is crated which tells BERT to only look the meaningful tokens and ignore tokens like [PAD].

Input text sequence:

```
LABEL: 0.0 (0=No Depression, 1=Depression)

ORIGINAL QUESTION:
why'd you move to l_a

ORIGINAL ANSWER:
um my family moved to the u_s and then i moved down here eventually for college

COMBINED INPUT STRING:
why'd you move to l_a [SEP] um my family moved to the u_s and then i moved down here eventually for college
```

Figure 1: Combination of sentences into one string

Conversion to tokens:

```
1. LIST OF TOKENS (Total 128):
['[CLS]', 'why', "'", 'd', 'you', 'move', 'to', 'l', '_', 'a', '[SEP]', 'um', 'my', 'family', 'moved', 'to', 'the', 'u', '_', 's', 'a
```

Figure 2: Conversion of text to tokens

[CLS] = Start of sequence token

[SEP] = Separator between question and answer

[PAD] = Padding to make the sequence 128 tokens long

Vector embeddings of token:

```
2. NUMERICAL INPUT_IDS (Total 128):
[ 101 2339 1005 1040 2017 2693 2000 1048 1035 1037  102 8529 2026 2155
 2333 2000 1996 1057 1035 1055 1998 2059 1045 2333 2091 2182 2776 2005
 2267  102    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0]
```

Figure 3: Conversion of tokens to input_IDs

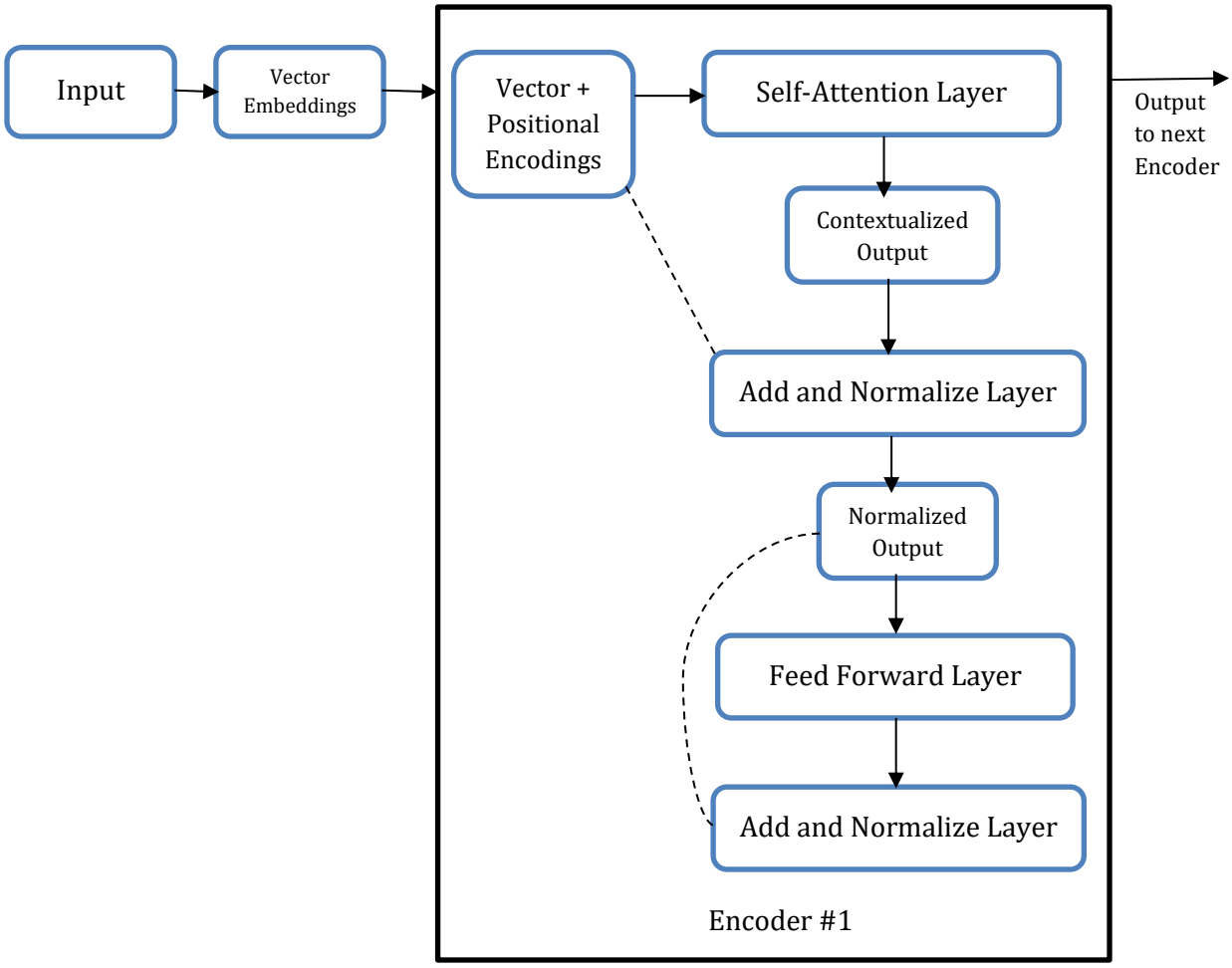
Attention mask:

```
3. ATTENTION_MASK (Total 128):
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

Figure 4: Attention mask of input tokens

1 for real tokens, 0 for padded tokens.

We have used the 'bert-base-uncased' model which consists of 12 encoders stacked on top of one another.



BERT Architecture

Figure 5: Architecture of BERT Model

In the Architecture the positional encoding is vector representing the *position* of each token is added to its embedding. It is added to help in cases when the vector embeddings is same but the context is different. The self-attention layer looks at all 128 tokens of our Q&A pair at the same time and learns the relationship between every token. This layer then outputs a new vectorhich is a contextualized version

the previous information. After this the original information is added back to the contextualized information so the original meaning is not lost. This new vector is then normalized. Feed Forward layer is a simple, standard two-layer neural network. It further processes each token's vector individually. It helps the model find more complex patterns within that specific token. The output is then fed into the next encoder till it passes through all 12 encoders. After this [CLS] is the final output vector 768-dimension summary of the entire Q&A pair. This final vector is passed to the linear layer. This layer contains its own small set of weights and a matrix that is 768 x 2 in size. This outputs the final vector which is passed through a SoftMax function to give the prediction. We use the AdamW optimizer with a learning rate of 2e-5 and weight decay. For validation we are using a 10-fold cross validation. In this the dataset is split 10 times and 10 separate models are trained on it and the final metrics are the average of these 10 models. This makes sure our results are stable and not a one-time fluke.

3.4 Audio Architecture

This section explains how we detect depression using audio recordings. We start with raw interview audio, clean it, convert it into spectrogram images, and then use a Vision Transformer (ViT) model to classify whether the speaker shows signs of depression. The entire project was done in Google Colab, and all data was stored and managed in Google Drive.

3.4.1 Data Processing Pipeline

We work with .wav audio files from the DIAC-WOZ dataset, which contains real clinical interview recordings.

These audio files are cleaned and converted into Mel-Spectrograms (images showing how sound changes over time).

3.4.1.1 Audio → Mel-Spectrogram Conversion

1. Load the file using Librosa
2. Remove background noise(fan noise, static, room echo)
3. Normalise the volume so all recordings are at the same loudness level
4. Convert it into a Mel-Spectrogram using a Short-Time Fourier Transform (STFT)

5. Convert the power values intro decibel(dB) scale to highlight the audio patterns clearly

A Mel-Spectrogram looks like a Heatmap:

Part of Spectrogram Meaning

X- axis time(speech progress)

Y-axis pitch/frequency

Part of Spectrogram Meaning

Colour Intensity how strong or weak the voice is

Each spectrogram is saved as a .png image and used as input to our model instead of using the raw audio.

The two below are just examples of the Spectrograms out of 189 images for audio data or the .wav file.

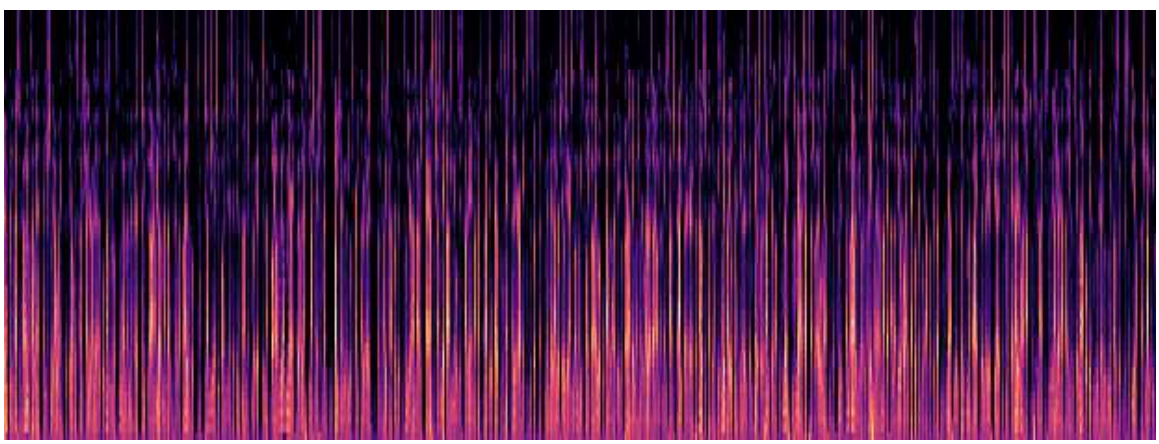


Figure 6: Sample spectrogram 1

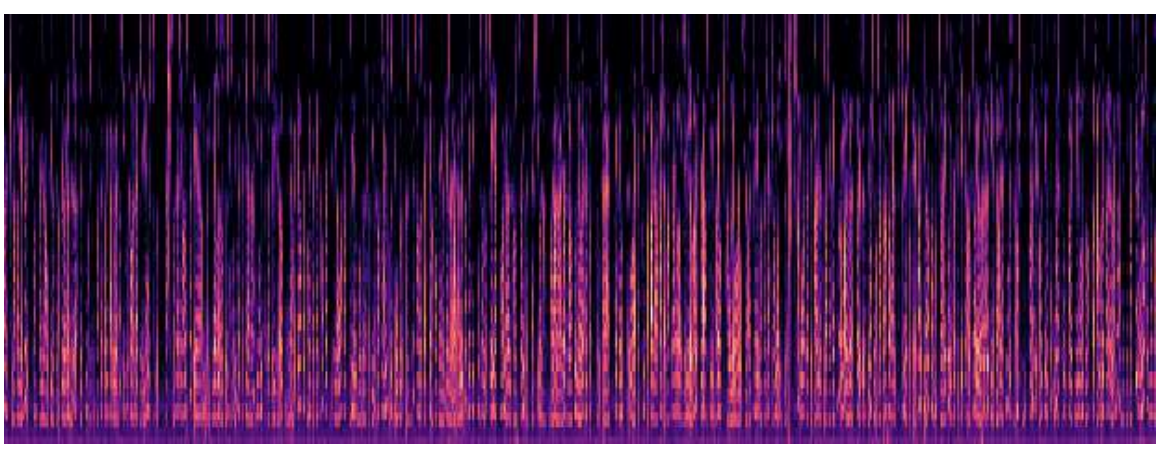


Figure 7: Sample spectrogram 2

3.4.1.2 Class Imbalance Handling

The dataset does not have an equal number of “Depressed” vs “Not Depressed” samples.

To prevent model bias, we use:

- Class weights -> model gives more importance to minority class
- Balanced sampling -> during training, both classes are shown fairly

This helps the model learn depression patterns instead of just predicting the majority class.

3.4.2 Workflow Overview

The workflow has three main steps:

1. Load and clean the audio
2. Convert audio to Mel-spectrograms
3. Train Visin Transformer model using these spectrograms

3.4.2.1 Visualizing Audio Signals

Before converting, we first plot the waveform of each audio file.

This helps us:

- Confirm the audio isn’t corrupted
- See if the person speaks slowly, quietly, or with long pauses(possible depression cues)

3.4.2.2 Power Spectrum Analysis

Next we analyse how the energy is distributed in different frequency ranges. People with depression often show:

- Lower speech energy

- Flatter tone

This analysis gives us early insight creating spectrograms.

3.4.2.3 Generating Mel-Spectrograms

We then generate Mel-Spectrograms with tuned parameters:

Parameter	What it does
Sample Rate(16kHz)	Standard setting for speech data
FFT size(n_fft)	Frequency Resolution
Hop Length	Controls smoothness of Spectrograms
Mel bins	Number of frequency ranges

All generated spectrograms are stored based on class (Depressed / Not_Depressed).

3.4.3 Organizing Spectrograms Into Classes

The folder structure looks like:

/Dataset/

    /Depressed/

    /Not Depressed/

This script automatically saves each spectrogram in the correct folder.

3.4.3.1 Automation Script

A Python script handles everything:

- Reads the audio
- Generates the spectrogram
- Saves it with proper naming format

It also prints progress, skip already processed files, and manages memory using:

Import gc

gc.collect()

3.4.4 Data Handling

We kept everything in Google Drive because:

- The dataset is large
- Colab resets RAM when disconnected
- Drive keep files persistent and organised

3.4.4.1 Google Drive Storage Setup

Both raw audio files and generated spectrograms are stored in Drive under separate folders.

3.5 Vision Transformer (Model)

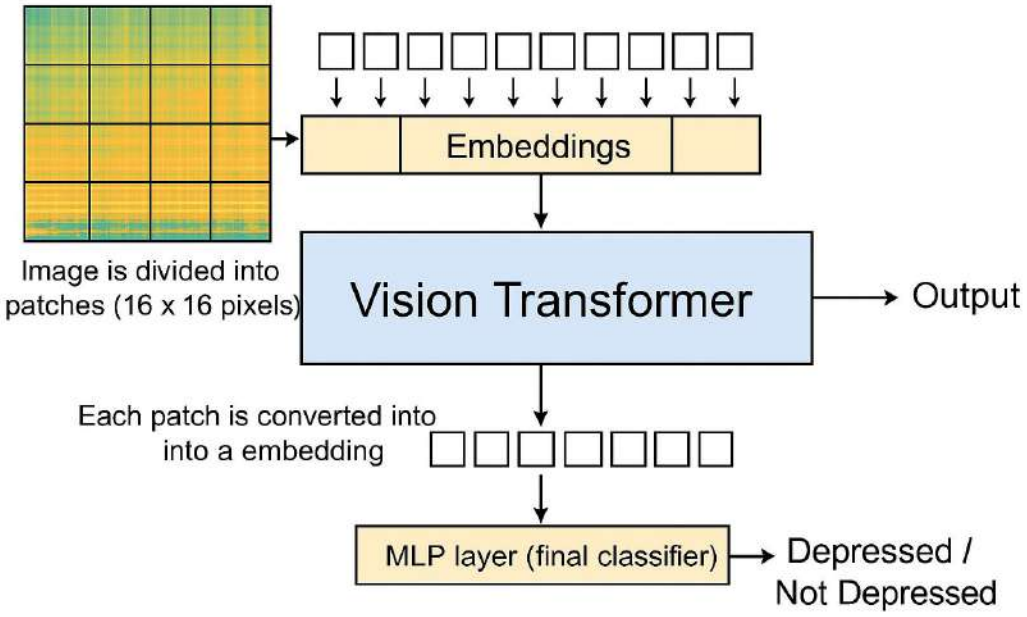


Figure 8: Vision Transformer Architecture

3.4.5.1 Preparing Data for ViT

Before fedding spectrograms to ViT, we:

- Resize them to 224 x 224
- Normalise pixel values
- Split data as:
  - 70% training
  - 15% validation
  - 15% testing

3.4.5.2 ViT Architecture (simplified)

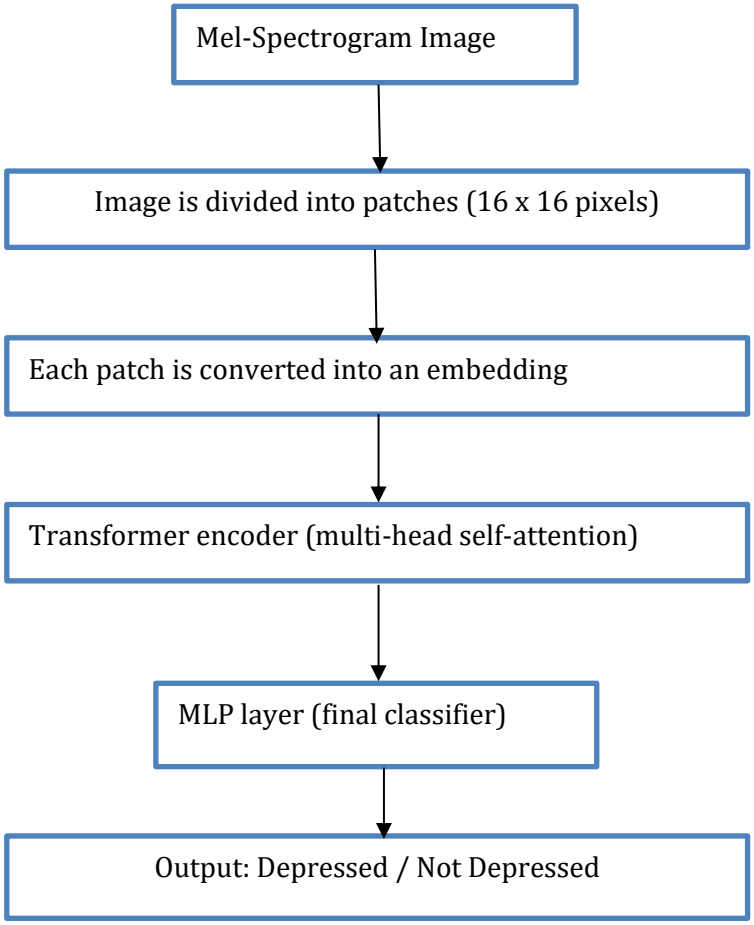


Figure 8: Simplified Vision Transformer Architecture

3.4.5.3 How ViT Works

Unlike CNNs, ViT:

- does NOT use convolution filters
- treats the spectrogram as a sequence of patches
- focuses on the entire image using self-attention

This helps ViT pick up signs like:

- long pauses
- low speech energy
- monotonous tone

—all of which are indicators of depression.

3.4.6 MLP (Final Layer)

After the transformer encoder processes the spectrogram, the MLP classifier predicts:

- 0 → Not Depressed
- 1 → Depressed

3.4.7 Data Format

All final spectrograms are stored in clearly labeled folders.
The model loads the data directly from these folders without manual labeling.

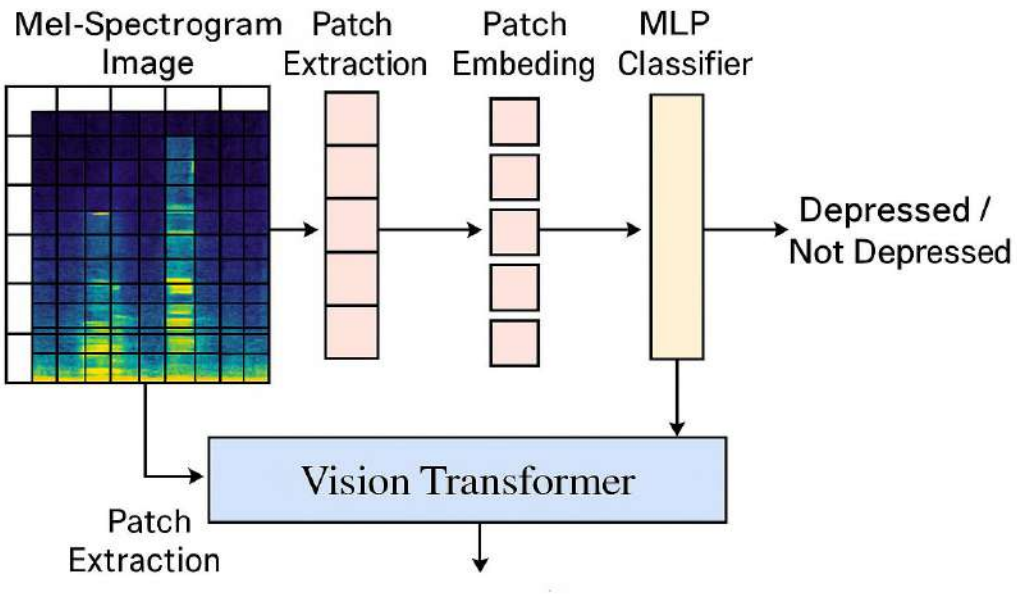


Figure 9: Vision Transformer Pipeline

Chapter 4 – Results & Discussion

4.1 Text-Based Model (BERT)

The fine-tuned BERT model was trained using the DAIC-WOZ transcript responses. Inputs were tokenized using the HuggingFace BERT tokenizer with a maximum sequence length of 128. The model was trained in Google Colab (T4 GPU), batch size of 16, and learning rate of 2e-5.

Training Summary:

- Model: bert-base-uncased
- Epochs: 10
- Optimizer: AdamW
- Loss function: Cross-Entropy
- Split: 80% Training, 10% Validation, 10% Test

The BERT model progressively improved during the training and converged without overfitting.

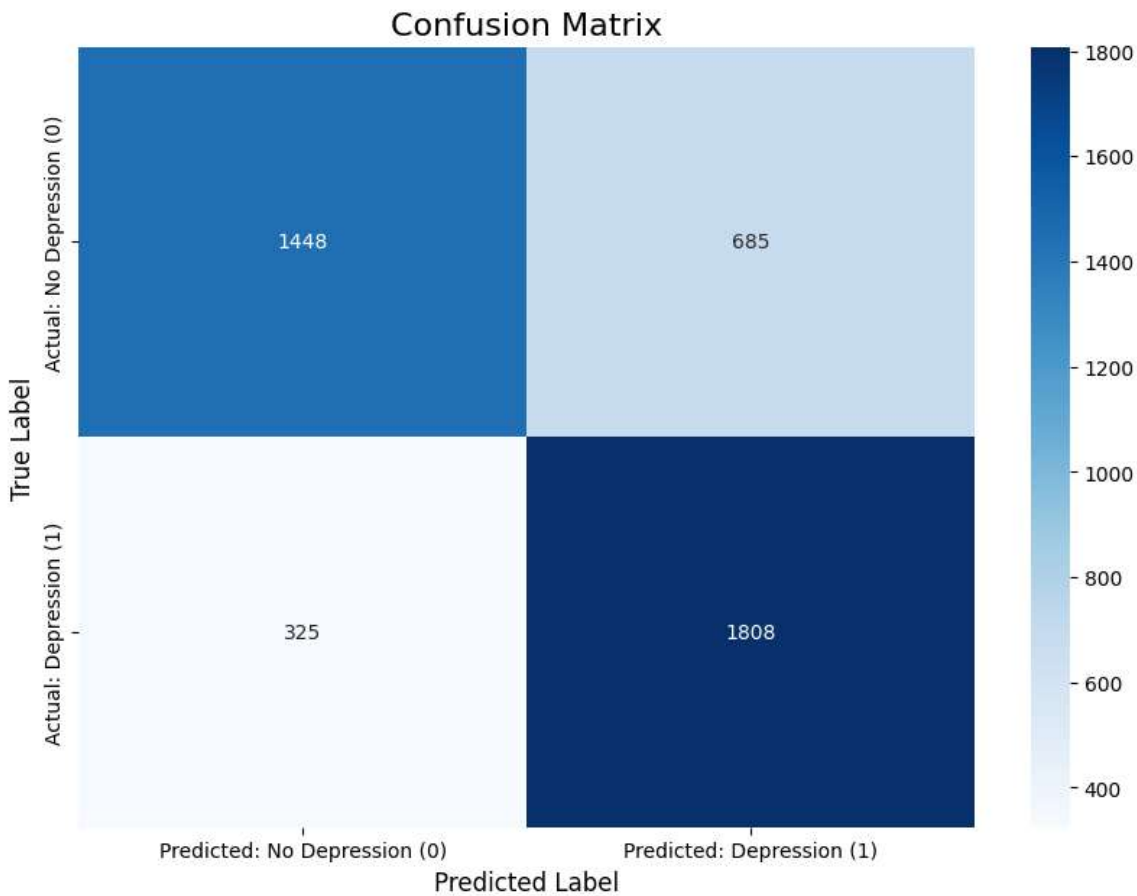


Figure 10: Confusion matrix for BERT

```
Average F1 Score: 0.7817 ± 0.0150
Average ROC-AUC: 0.8375 ± 0.0173
Average Accuracy: 0.7633 ± 0.0183
```

Figure 11: Performance metrics for BERT

Our BERT model used for text classification had a final accuracy of 0.7633 ± 0.0183.

4.2 Audio-Based Model (Vision Transformer – ViT)

For the audio modality first, our raw audio was denoised using a band-pass filter. The cleaned audio was then converted into Mel-spectrograms, resized into patches and then fed to the Vison Transformer.

Training Summary:

- Model: ViT-Base Patch16
- Epochs: 20
- Optimizer: AdamW
- Loss Function: Cross-Entropy
- Split: 70% Training, 15% Validation, 15% Test

```
Training done in 27.0 min. Best Val Acc=0.690
✅ Embeddings saved at /content/drive/MyDrive/DDS/ViT_Results/embeddings.npy (shape (142, 197, 768))
🎯 Final overall accuracy: 0.704225352112676
✅ All results stored in: /content/drive/MyDrive/DDS/ViT_Results
```

Figure 12: Accuracy for ViT

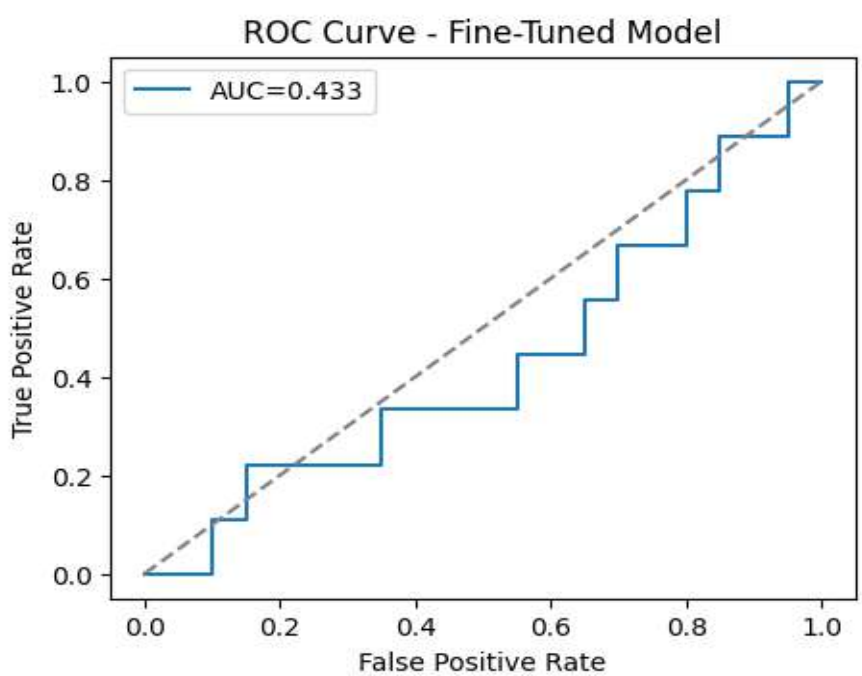


Figure 13: ROC Curve for ViT

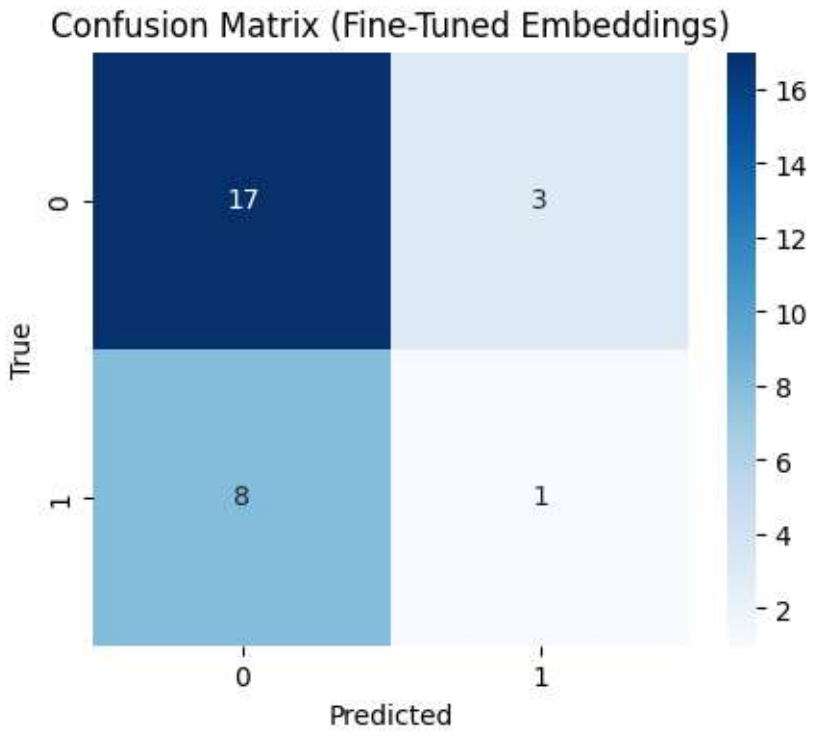


Figure 14: Confusion Matrix for ViT

The ViT audio classifier achieved a final accuracy of 69%.

4.3 Comparison With Previous Work

4.3.1 Text Modality

Study	**Model**	**F1-score**
Petrov et al	RoBERTa (Hierarchical Regression)	0.739
Chi et al	RoBERTa (Participant only)	0.75
Mustafa et al	DepRoBERTa	0.71
Zhang et al	Multi-MT5 + Multi RoBERTa	0.88
Our Work	BERT	**0.78**

Table 4: Comparison of text model with related studies

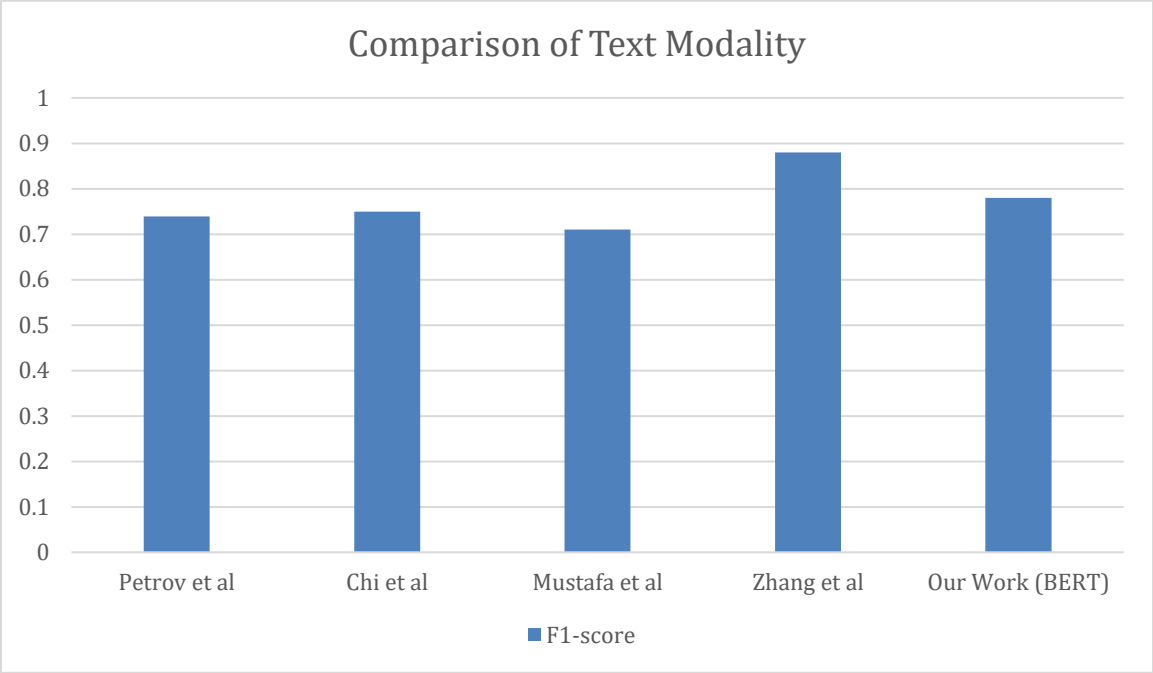


Figure 15: Graph comparing BERT to other models

Our BERT model is on par with and in some case outperforms other similar models only being beaten by specialized multi-model like that of Zhang et al. This shows that BERT is a good fit for the task of text classification.

4.3.2 Audio Modality

Study	**Model**	**Accuracy**
Milo et al	CNN (Baseline for SNN conversion)	82.51%
Zhou et al	CNN (Handcrafted features)	83%
Ravi et al	DepAudioNet	74%
Feng et al	CNN (Acoustic features)	77%
Our Work	ViT	**69%**

Table 5: Comparison of audio model with related studies

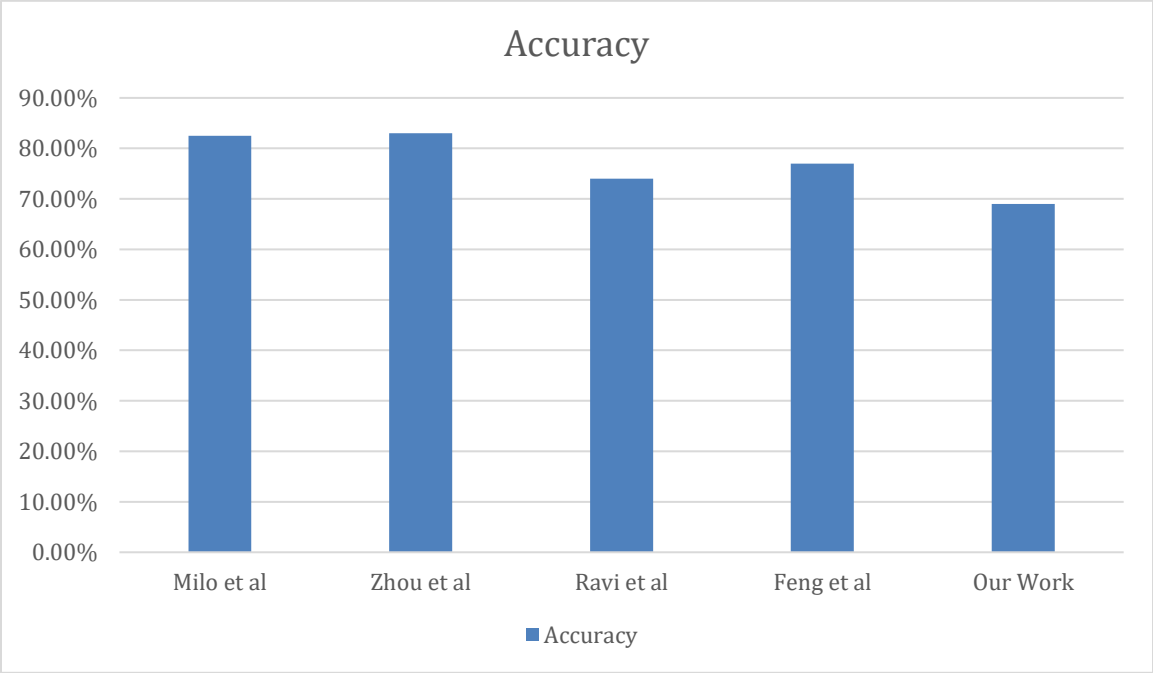


Figure 16: Graph comparing ViT to other models

Our ViT model is unable to be on par with other models such as traditional CNN based pipelines. This shows that Vision Transformers are not fir for the task of audio classification for depression detection.

4.4 Discussion

- Both text and speech-based models offer different benefits in terms of depression detection.
- Text models capture linguistic cues such as the use and frequency of negative words, self-referential language, etc
- While audio models analyse vocal features like tone, pitch, rhythm and speech hesitations which may give away things that a person does not verbally express.
- Our text model BERT is on par with currently used methods which shows it is a good fit for text-based depression detection

- Our audio model ViT underperformed which goes to show that audio-based depression detection might be a task it might be unsuited to.

Chapter 5 – Conclusion & Future Scope

5.1 Conclusion

This project successfully demonstrates the feasibility of using deep learning techniques to detect depression using both speech and text modalities. With the help of the DAIC-WOZ clinical dataset, we trained two transformer-based models:

- **BERT (Text-based model)**: It achieved an F1-score of 0.78 which shows that it was successful in capturing linguistic cues such as negative sentiment, shifts in emotional tone, etc. It showed that text based assessment can provide evidence of underlying mood disorders
- **Vision Transformer (Audio-based model)**: It achieved an accuracy of 69%, showing that acoustic speech patterns (monotonic pitch, reduced loudness, longer pauses) correlate with depressive symptoms. But it did not come close to other more state of the art models, showing that perhaps ViT is not suited for this task.

The findings reinforce studies which state that depression in individuals can be diagnosed using linguistic and acoustic patterns. Hence deep learning models can help create tools to better diagnose depression at early stages

Overall, the project meets all research objectives:

- Develop two deep learning pipelines (text + audio) which were designed and evaluated.
- Our results showed BERT > ViT for single-modality depression detection.
- Literature comparison confirmed competitive performance.

Hence, Deep learning + transformer architectures can assist mental health screening and provide early intervention support.

5.2 Future Scope

The project can be extended in several meaningful directions:

1. **Multimodal Fusion**
    - Combine BERT text embeddings + ViT audio embeddings to make a early fusion multimodal model.
    - This may improve the accuracy beyond what both a separately achieved >76%
2. **Real-time Depression Detection System**
    - Use this model in a smartphone/web app
    - This can be then used to analyse speech and chat conversations non-intrusively
3. **LLM Clinical Explanation**
    - Integrate explainability (XAI) to show *why* our model has classified a person as depressed
    - Instead of our model being a black box which one can't know why it made a certain prediction we will know exactly why it made a certain prediction which will help it become trustworthy in medical fields
4. **Dataset Expansion**
    - Collect and include interviews from other multilingual speech (Hindi, Tamil, etc.) and regional accents

Appendix

Code – Text Pipeline (BERT Fine-Tuning)

Data Preprocessing

```python
import pandas as pd
import os
import glob
```

```python
# --- 1. SETUP ---

path_to_files = 'data/*.csv'

INTERROGATIVE_WORDS = (
    "what", "when", "where", "which", "who", "whom", "whose", "why", "how",
    "is", "are", "am", "was", "were",
    "do", "does", "did",
    "has", "have", "had",
    "can", "could", "will", "would", "should", "may", "might",
    "tell me", "describe", "explain", "give me an example",
    "so,", "and what", "what about", "how about"
)
```

```python
all_qa_data = []

# --- 2. PROCESS EACH FILE ---

transcript_files = glob.glob(path_to_files)

if not transcript_files:
    print(f"Error: No files found at '{path_to_files}'. Please check the path.")
else:
    for file_path in transcript_files:
        filename = os.path.basename(file_path)
        person_id = filename.split('_')[0]

        try:
            df = pd.read_csv(file_path, sep='\t', names=['Speaker', 'value'], header=0)

            current_question = ""
            current_answer_parts = []

            for index, row in df.iterrows():
                speaker = str(row['Speaker']).strip()
                dialogue = str(row['value']).strip()

                if speaker == 'Ellie':
                    if current_question and current_answer_parts:
                        full_answer = " ".join(current_answer_parts)
                        all_qa_data.append({
                            'personId': person_id,
                            'question': current_question,
                            'answer': full_answer
                        })

                    current_question = dialogue
                    current_answer_parts = []
```

```python
                elif speaker == 'Participant':
                    if current_question and dialogue:
                        current_answer_parts.append(dialogue)
            if current_question and current_answer_parts:
                full_answer = " ".join(current_answer_parts)
                all_qa_data.append({
                    'personId': person_id,
                    'question': current_question,
                    'answer': full_answer
                })

        except Exception as e:
            print(f"Could not process file {file_path}. Error: {e}")
```

```python
if all_qa_data:
    # --- 3. CLEAN AND FILTER THE DATA ---
    final_df = pd.DataFrame(all_qa_data)


    final_df.dropna(subset=['question', 'answer'], inplace=True)
    final_df = final_df[final_df['question'].str.strip() != '']
    final_df = final_df[final_df['answer'].str.strip() != '']

    final_df = final_df[final_df['question'].str.lower().str.startswith(INTERROGATIVE_WORDS)]

    # --- 4. DISPLAY AND SAVE THE RESULT ---
    print("--- Processed Question-Answer DataFrame ---")
    print(final_df.head())
    print(f"\nTotal pairs extracted: {len(final_df)}")

    final_df.to_csv('processed_qa_pairs.csv', index=False)
    print("\nSaved the final results to 'processed_qa_pairs.csv'")
```

Python

```
--- Processed Question-Answer DataFrame ---
  personId                                   question  \
0      300                        how are you doing today
1      300                   where are you from originally
2      300                          why'd you move to l_a
3      300                            how do you like l_a
4      300  what are some things you really like about l_a

                                          answer
0                                           good
1                                  atlanta georgia
2                    um my parents are from here um
3                                       i love it
4  i like the weather i like the opportunities um...

Total pairs extracted: 3575

Saved the final results to 'processed_qa_pairs.csv'
```

```python
try:
    final_df['personId'] = final_df['personId'].astype(int)
    labels_df['Participant_ID'] = labels_df['Participant_ID'].astype(int)
    print("Successfully converted ID columns to integer type.")
except ValueError as e:
    print(f"Error converting columns to integer: {e}")
    print("Please check your ID columns for non-numeric values.")
    exit()
```

Python

```
Successfully converted ID columns to integer type.
```

```python
final_df_with_labels = pd.merge(
    final_df,
    labels_df,
    left_on='personId',
    right_on='Participant_ID',
    how='inner'
)
```

Python

```python
Click to add a breakpoint s.drop('Participant_ID', axis=1, inplace=True)

print("\n--- Merged DataFrame with PHQ_Binary labels ---")
print(final_df_with_labels.head())
```

Python

```
--- Merged DataFrame with PHQ_Binary labels ---
   personId                                question  \
0       300                     how are you doing today
1       300                where are you from originally
2       300                        why'd you move to l_a
3       300                          how do you like l_a
4       300  what are some things you really like about l_a

                                        answer  PHQ_Binary
0                                         good         0.0
1                                atlanta georgia         0.0
2                   um my parents are from here um         0.0
3                                      i love it         0.0
4  i like the weather i like the opportunities um...         0.0
```

```python
final_df_with_labels.to_csv('data_with_labels.csv', index=False)
```

Python

Data Balancing

```python
import pandas as pd
from sklearn.utils import resample
```

Python

```python
INPUT_FILENAME = 'data_with_labels.csv'

LABEL_COLUMN_NAME = 'PHQ_Binary'

OUTPUT_FILENAME = 'balanced_data.csv'
```

Python

```python
# --- 1. LOAD THE DATASET ---

try:
    data_df = pd.read_csv(INPUT_FILENAME)
    print(f"Successfully loaded '{INPUT_FILENAME}'.")
    print(f"Total samples found: {len(data_df)}")
except FileNotFoundError:
    print(f"Error: The file '{INPUT_FILENAME}' was not found.")
    print("Please make sure the csv file is in the same directory as this script.")
    exit()
```

Python

```
Successfully loaded 'data_with_labels.csv'.
Total samples found: 3575
```

```python
# --- 2. SEPARATE THE MAJORITY AND MINORITY CLASSES ---

df_majority = data_df[data_df[LABEL_COLUMN_NAME] == 0]
df_minority = data_df[data_df[LABEL_COLUMN_NAME] == 1]

print("\n--- Original Class Distribution ---")
print(f"Non-depression (majority) samples: {len(df_majority)}")
print(f"Depression (minority) samples: {len(df_minority)}")
```

Python

```
--- Original Class Distribution ---
Non-depression (majority) samples: 2133
Depression (minority) samples: 1442
```

```python
# --- 3. OVERSAMPLE THE MINORITY CLASS ---

df_minority_upsampled = resample(df_minority,
                                 replace=True,
                                 n_samples=len(df_majority),
                                 random_state=123)

print(f"\nUpsampled depression samples to match majority: {len(df_minority_upsampled)}")
```

Python

```
Upsampled depression samples to match majority: 2133
```

```python
# --- 4. COMBINE AND SHUFFLE THE DATAFRAME ---

df_balanced = pd.concat([df_majority, df_minority_upsampled])

df_balanced = df_balanced.sample(frac=1, random_state=42).reset_index(drop=True)
```

Python

```python
# --- 5. VERIFY AND SAVE THE FINAL BALANCED DATAFRAME ---

print("\n--- Balanced Dataset Verification ---")
print(f"Total samples in the new balanced dataset: {len(df_balanced)}")
print("Final class distribution:")
print(df_balanced[LABEL_COLUMN_NAME].value_counts())
```

Python

```
--- Balanced Dataset Verification ---
Total samples in the new balanced dataset: 4266
Final class distribution:
PHQ_Binary
1.0    2133
0.0    2133
Name: count, dtype: int64
```

```python
df_balanced.to_csv(OUTPUT_FILENAME, index=False)
```

Python

BERT Model

```python
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import BertTokenizer, BertForSequenceClassification, get_linear_schedule_with_warmup
from torch.optim import AdamW
from sklearn.model_selection import KFold
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix
import numpy as np
```

```python
# --- Configuration and Hyperparameters ---
MODEL_NAME = 'bert-base-uncased'
INPUT_FILENAME = 'balanced_data.csv'
TEXT_COLUMN = 'text'
LABEL_COLUMN = 'PHQ_Binary'
```

```python
# Hyperparameters
MAX_LEN = 128
BATCH_SIZE = 8
EPOCHS = 10
LEARNING_RATE = 2e-5
WEIGHT_DECAY = 0.01
NUM_WORKERS = 2
```

```python
N_SPLITS = 10
```

```python
# --- 1. Load Full Dataset ---
print("Loading and preparing data...")
df_full = pd.read_csv(INPUT_FILENAME)
df_full[TEXT_COLUMN] = df_full['question'] + " [SEP] " + df_full['answer']
```

```
Loading and preparing data...
```

```python
# --- 2. Custom PyTorch Dataset ---
class DepressionDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, item):
        text = str(self.texts[item])
        label = self.labels[item]

        encoding = self.tokenizer.encode_plus(
            text,
            add_special_tokens=True,
            max_length=self.max_len,
            return_token_type_ids=False,
            padding='max_length',
            truncation=True,
            return_attention_mask=True,
            return_tensors='pt',
        )


        return {
            'text': text,
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(label, dtype=torch.long)
        }
```

```python
# --- 3. Dataloader Function ---
tokenizer = BertTokenizer.from_pretrained(MODEL_NAME)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

def create_data_loader(df, tokenizer, max_len, batch_size):
    ds = DepressionDataset(
        texts=df[TEXT_COLUMN].to_numpy(),
        labels=df[LABEL_COLUMN].to_numpy(),
        tokenizer=tokenizer,
        max_len=max_len
    )
    return DataLoader(
        ds,
        batch_size=batch_size,
        num_workers=NUM_WORKERS
    )
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 6.02kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 2.65MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 5.91MB/s]
config.json: 100%  570/570 [00:00<00:00, 24.6kB/s]
Using device: cuda
```

```python
# --- 4. Training and Evaluation Functions ---
def train_epoch(model, data_loader, optimizer, device, scheduler):
    model = model.train()
    losses = []
    for d in data_loader:
        input_ids = d["input_ids"].to(device)
        attention_mask = d["attention_mask"].to(device)
        labels = d["labels"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        loss = outputs.loss
        losses.append(loss.item())

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()

    return np.mean(losses)
```

```python
def eval_model(model, data_loader, device):
    model = model.eval()
    all_labels = []
    all_probs = []

    with torch.no_grad():
        for d in data_loader:
            input_ids = d["input_ids"].to(device)
            attention_mask = d["attention_mask"].to(device)
            labels = d["labels"].to(device)

            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )
            logits = outputs.logits

            probs = torch.softmax(logits, dim=1)[:, 1].cpu().numpy()
            all_probs.extend(probs)
            all_labels.extend(labels.cpu().numpy())

    predictions = (np.array(all_probs) >= 0.5).astype(int)
    return {
        'accuracy': accuracy_score(all_labels, predictions),
        'f1': f1_score(all_labels, predictions),
        'roc_auc': roc_auc_score(all_labels, all_probs),
        'labels': all_labels,
        'preds': predictions
    }
```

```python
# --- 5. Main K-Fold Cross-Validation Loop ---
def run_kfold():
    kfold = KFold(n_splits=N_SPLITS, shuffle=True, random_state=42)

    fold_f1_scores = []
    fold_roc_auc_scores = []
    fold_accuracy_scores = []

    master_labels = []
    master_preds = []

    for fold, (train_ids, test_ids) in enumerate(kfold.split(df_full)):
        print(f'--- Fold {fold + 1}/{N_SPLITS} ---')

        model = BertForSequenceClassification.from_pretrained(
            MODEL_NAME,
            num_labels=2
        )
        model = model.to(device)

        df_train = df_full.iloc[train_ids]
        df_test = df_full.iloc[test_ids]

        train_data_loader = create_data_loader(df_train, tokenizer, MAX_LEN, BATCH_SIZE)
        test_data_loader = create_data_loader(df_test, tokenizer, MAX_LEN, BATCH_SIZE)
```

```python
        optimizer = AdamW(
            model.parameters(),
            lr=LEARNING_RATE,
            weight_decay=WEIGHT_DECAY
        )
        total_steps = len(train_data_loader) * EPOCHS
        scheduler = get_linear_schedule_with_warmup(
            optimizer,
            num_warmup_steps=0,
            num_training_steps=total_steps
        )

        for epoch in range(EPOCHS):
            print(f'  Epoch {epoch + 1}/{EPOCHS}')
            train_loss = train_epoch(model, train_data_loader, optimizer, device, scheduler)
            if train_loss is None:
                print("  Training step failed, moving to next fold.")
                break
            print(f'  Training loss: {train_loss:.4f}')

        if train_loss is None:
            continue

        print(f'  Evaluating fold {fold + 1}...')
        test_metrics = eval_model(model, test_data_loader, device)

        print(f"  Fold {fold + 1} F1: {test_metrics['f1']:.4f} | ROC-AUC: {test_metrics['roc_auc']:.4f} | Acc: {test_metrics['accuracy']:.4f}")
```

```python
        fold_f1_scores.append(test_metrics['f1'])
        fold_roc_auc_scores.append(test_metrics['roc_auc'])
        fold_accuracy_scores.append(test_metrics['accuracy'])

        master_labels.extend(test_metrics['labels'])
        master_preds.extend(test_metrics['preds'])

        # --- G. Calculate and Print Final Average Metrics ---
        print("\n--- K-Fold Cross-Validation Complete ---")
        if fold_f1_scores: # Check if any folds actually completed
            print(f'Average F1 Score: {np.mean(fold_f1_scores):.4f} \u0081 {np.std(fold_f1_scores):.4f}')
            print(f'Average ROC-AUC: {np.mean(fold_roc_auc_scores):.4f} \u0081 {np.std(fold_roc_auc_scores):.4f}')
            print(f'Average Accuracy: {np.mean(fold_accuracy_scores):.4f} \u0081 {np.std(fold_accuracy_scores):.4f}')

        else:
            print("No folds were successfully completed. Please check errors.")
```

```python
if __name__ == '__main__':
    run_kfold()
```

```python
if __name__ == '__main__':
    run_kfold()
```

```
--- Fold 1/10 ---
model.safetensors:  0%|          | 0.00/440M [00:00<?, ?B/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
  Epoch 1/10
  Training Loss: 0.6868
  Epoch 2/10
  Training Loss: 0.5751
  Epoch 3/10
  Training Loss: 0.3026
  Epoch 4/10
  Training Loss: 0.2618
  Epoch 5/10
  Training Loss: 0.2013
  Epoch 6/10
  Training Loss: 0.1589
  Epoch 7/10
  Training Loss: 0.1220
  Epoch 8/10
  Training Loss: 0.1017
  Epoch 9/10
  Training Loss: 0.0879
  Epoch 10/10
  Training Loss: 0.0801
  Evaluating Fold 1...
  Fold 1 F1: 0.7609 | ROC_AUC: 0.8068 | Acc: 0.7411

--- K-Fold Cross-Validation Complete ---
Average F1 Score: 0.7609 ± 0.0000
Average ROC-AUC: 0.8068 ± 0.0000
Average Accuracy: 0.7411 ± 0.0000

--- Aggregate Confusion Matrix (All Folds) ---

         Predicted 0     Predicted 1
        (No Depression)  (Depression)
Actual 0   140           74
(No Depression)
Actual 1   23            185
(Depression)

True Negatives (TN): 140
False Positives (FP): 74
False Negatives (FN): 23
True Positives (TP): 185
--- Fold 2/10 ---
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
  Epoch 1/10
  Training Loss: 0.0002
  Epoch 2/10
  Training Loss: 0.6219
  Epoch 3/10
  Training Loss: 0.4336
  Epoch 4/10
```

```
  Training Loss: 0.6662
  Epoch 1/10
  Training Loss: 0.6219
  Epoch 2/10
  Training Loss: 0.4336
  Epoch 4/10
  Training Loss: 0.3940
  Epoch 5/10
  Training Loss: 0.3017
  Epoch 6/10
  Training Loss: 0.2550
  Epoch 7/10
  Training Loss: 0.1423
  Epoch 8/10
  Training Loss: 0.0959
  Epoch 9/10
  Training Loss: 0.0949
  Epoch 10/10
  Training Loss: 0.0910
  Evaluating Fold 2...
  Fold 2 F1: 0.7628 | ROC_AUC: 0.8344 | Acc: 0.7471

--- K-Fold Cross-Validation Complete ---
Average F1 Score: 0.7708 ± 0.0109
Average ROC-AUC: 0.8222 ± 0.0135
Average Accuracy: 0.7541 ± 0.0076

--- Aggregate Confusion Matrix (All Folds) ---

         Predicted 0     Predicted 1
        (No Depression)  (Depression)
Actual 0   206           142
(No Depression)
Actual 1   68            358
(Depression)

True Negatives (TN): 206
False Positives (FP): 142
False Negatives (FN): 68
True Positives (TP): 358
--- Fold 3/10 ---
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
  Epoch 1/10
  Training Loss: 0.6994
  Epoch 2/10
  Training Loss: 0.6483
  Epoch 3/10
  Training Loss: 0.4684
  Epoch 4/10
  Training Loss: 0.3536
  Epoch 5/10
  Training Loss: 0.2274
  Epoch 6/10
  Training Loss: 0.3668
```

```
--- Fold 10/10 ---
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably train this model on a down-stream task to be able to use it for predictions and inference.
  Epoch 1/10
  Training Loss: 0.6911
  Epoch 2/10
  Training Loss: 0.6876
  Epoch 3/10
  Training Loss: 0.4366
  Epoch 4/10
  Training Loss: 0.2991
  Epoch 5/10
  Training Loss: 0.2144
  Epoch 6/10
  Training Loss: 0.1605
  Epoch 7/10
  Training Loss: 0.1310
  Epoch 8/10
  Training Loss: 0.1151
  Epoch 9/10
  Training Loss: 0.0994
  Epoch 10/10
  Training Loss: 0.0760
  Evaluating Fold 10...
  Fold 10 F1: 0.7812 | ROC-AUC: 0.8363 | Acc: 0.7700

--- K-Fold Cross-Validation Complete ---
Average F1 Score: 0.7817 ± 0.0150
Average ROC-AUC: 0.8375 ± 0.0173
Average Accuracy: 0.7633 ± 0.0183

--- Aggregate Confusion Matrix (All Folds) ---

          Predicted 0        Predicted 1
          (No Depression)   (Depression)
Actual 0    1448            685
(No Depression)
Actual 1    325             1808
(Depression)

True Negatives (TN): 1448
False Positives (FP): 685
False Negatives (FN): 325
True Positives (TP): 1808
```

```
--- K-Fold Cross-Validation Complete ---
Average F1 Score: 0.7817 ± 0.0150
Average ROC-AUC: 0.8375 ± 0.0173
Average Accuracy: 0.7633 ± 0.0183

--- Aggregate Confusion Matrix (All Folds) ---

          Predicted 0        Predicted 1
          (No Depression)   (Depression)
Actual 0    1448            685
(No Depression)
Actual 1    325             1808
(Depression)

True Negatives (TN): 1448
False Positives (FP): 685
False Negatives (FN): 325
True Positives (TP): 1808
```

Code – Audio Pipeline (ViT Spectrogram Classification)

Data Preprocessing

```python
# 🧠 STEP 1: Install dependencies
!pip install pydub
!apt-get install -y ffmpeg

# 🧠 STEP 2: Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

```
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packages (0.25.1)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ffmpeg is already the newest version (7:4.4.2-0ubuntu0.22.04.1).
0 upgraded, 0 newly installed, 0 to remove and 41 not upgraded.
Mounted at /content/drive
```

```python
# 🧠 STEP 3: Import libraries and define paths
import os
import pandas as pd
from pydub import AudioSegment

# Main folders (edit if your folder name differs)
base_dir = "/content/drive/MyDrive/DDS"
audio_dir = os.path.join(base_dir, "AUDIO")
data_dir = os.path.join(base_dir, "DATAS")
output_dir = os.path.join(base_dir, "cleaned_audio")
os.makedirs(output_dir, exist_ok=True)

print("Base folder:", base_dir)
print("Audio folder:", audio_dir)
print("Transcript folder:", data_dir)
print("Output folder:", output_dir)
```

```
/usr/local/lib/python3.12/dist-packages/pydub/utils.py:300: SyntaxWarning: invalid escape sequence '\('
  m = re.match('([su]([0-9]{1,2})p?) \(([0-9]{1,2}) bit\)$', token)
/usr/local/lib/python3.12/dist-packages/pydub/utils.py:301: SyntaxWarning: invalid escape sequence '\('
  m2 = re.match('([su]([0-9]{1,2})p?)( \(default\))?$', token)
/usr/local/lib/python3.12/dist-packages/pydub/utils.py:310: SyntaxWarning: invalid escape sequence '\('
  elif re.match('(flt)p?( \(default\))?$', token):
/usr/local/lib/python3.12/dist-packages/pydub/utils.py:314: SyntaxWarning: invalid escape sequence '\('
  elif re.match('(dbl)p?( \(default\))?$', token):
Base folder: /content/drive/MyDrive/DDS
Audio folder: /content/drive/MyDrive/DDS/AUDIO
Transcript folder: /content/drive/MyDrive/DDS/DATAS
Output folder: /content/drive/MyDrive/DDS/cleaned_audio
```

```python
# 🧠 STEP 3: Import and set paths
import os
import re
import pandas as pd
from pydub import AudioSegment

base_dir = "/content/drive/MyDrive/DDS"
audio_dir = os.path.join(base_dir, "AUDIO")
data_dir = os.path.join(base_dir, "DATAS")
output_dir = os.path.join(base_dir, "cleaned_audio")
os.makedirs(output_dir, exist_ok=True)
```

```python
# 🧠 STEP 4: Helper function to extract ID number (e.g. "300" from "300_TRANSCRIPT_P.csv")
def extract_id(filename):
    match = re.match(r"(\d+)", filename)
    return match.group(1) if match else None
```

```python
# 🧠 STEP 5: Process each transcript and match to corresponding audio
for file in os.listdir(data_dir):
    if file.endswith(".csv"):
        transcript_id = extract_id(file)
        if not transcript_id:
            print(f"⚠️ Skipping {file}: could not extract ID.")
            continue

        # Find matching audio file by ID
        matching_audio = [a for a in os.listdir(audio_dir) if transcript_id in a and a.endswith(".wav")]
        if not matching_audio:
            print(f"⚠️ No matching audio found for {file}")
            continue

        wav_path = os.path.join(audio_dir, matching_audio[0])
        csv_path = os.path.join(data_dir, file)
        out_path = os.path.join(output_dir, f"{transcript_id}_cleaned.wav")

        # Load transcript
        df = pd.read_csv(csv_path)
        participant_df = df[df["speaker"] != "Ellie"]
```

```python
        # Keep only Participant dialogues
        participant_df = df[df["speaker"] == "Participant"]

        # Load full audio
        audio = AudioSegment.from_wav(audio_path)

        # Concatenate participant segments
        participant_audio = AudioSegment.empty()
        for _, row in participant_df.iterrows():
            start_ms = row["start_time"] * 1000
            end_ms = row["stop_time"] * 1000
            participant_audio += audio[start_ms:end_ms]

        # Export result
        out_path = os.path.join(output_dir, f"{transcript_id}_Participant.wav")
        participant_audio.export(out_path, format="wav")
        print(f"✅ Saved: {out_path}")
```

```
•••  Processing: 327_TRANSCRIPT.csv with 327_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/327_Participant.wav
     Processing: 326_TRANSCRIPT.csv with 326_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/326_Participant.wav
     Processing: 328_TRANSCRIPT.csv with 328_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/328_Participant.wav
     Processing: 329_TRANSCRIPT.csv with 329_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/329_Participant.wav
     Processing: 330_TRANSCRIPT.csv with 330_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/330_Participant.wav
     Processing: 331_TRANSCRIPT.csv with 331_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/331_Participant.wav
     Processing: 332_TRANSCRIPT.csv with 332_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/332_Participant.wav
     Processing: 333_TRANSCRIPT.csv with 333_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/333_Participant.wav
     Processing: 334_TRANSCRIPT.csv with 334_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/334_Participant.wav
     Processing: 335_TRANSCRIPT.csv with 335_AUDIO.wav
     ✅ Saved: /content/drive/MyDrive/DDS/Cleaned_Audio/335_Participant.wav
     Processing: 336_TRANSCRIPT.csv with 336_AUDIO.wav
```

```python
import os
import librosa
import noisereduce as nr
import soundfile as sf

# Define directories
base_dir = "/content/drive/MyDrive/DDS"
input_dir = os.path.join(base_dir, "Cleaned_Audio")
output_dir = os.path.join(base_dir, "Denoised_Audio")
os.makedirs(output_dir, exist_ok=True)

# Loop through each cleaned file
for file in os.listdir(input_dir):
    if file.endswith(".wav"):
        input_path = os.path.join(input_dir, file)
        output_path = os.path.join(output_dir, file.replace(".wav", "_denoised.wav"))

        print(f"🎧 Denoising {file} ...")

        # Load audio
        y, sr = librosa.load(input_path, sr=None)

        # Estimate noise profile (first 0.5 seconds or so)
        noise_sample = y[:int(0.5 * sr)]

        # Apply noise reduction
```

```python
        # Apply noise reduction
        reduced_audio = nr.reduce_noise(y=y, y_noise=noise_sample, sr=sr, prop_decrease=0.95)

        # Save denoised audio
        sf.write(output_path, reduced_audio, sr)
        print(f"✅ Saved: {output_path}")

print("🎉 All files denoised successfully!")
```

```
🎧 Denoising 440_Participant.wav ...
✅ Saved: /content/drive/MyDrive/DDS/Denoised_Audio/440_Participant_denoised.wav
🎧 Denoising 431_Participant.wav ...
✅ Saved: /content/drive/MyDrive/DDS/Denoised_Audio/431_Participant_denoised.wav
🎧 Denoising 442_Participant.wav ...
✅ Saved: /content/drive/MyDrive/DDS/Denoised_Audio/442_Participant_denoised.wav
🎧 Denoising 321_Participant.wav ...
✅ Saved: /content/drive/MyDrive/DDS/Denoised_Audio/321_Participant_denoised.wav
🎧 Denoising 320_Participant.wav ...
✅ Saved: /content/drive/MyDrive/DDS/Denoised_Audio/320_Participant_denoised.wav
🎧 Denoising 427_Participant.wav ...
✅ Saved: /content/drive/MyDrive/DDS/Denoised_Audio/427_Participant_denoised.wav
🎧 Denoising 433_Participant.wav ...
✅ Saved: /content/drive/MyDrive/DDS/Denoised_Audio/433_Participant_denoised.wav
🎉 All files denoised successfully!
```

Spectrogram Generation

```python
import os
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np

# Define directories
base_dir = "/content/drive/MyDrive/DDS"
input_dir = os.path.join(base_dir, "Denoised_Audio")
output_dir = os.path.join(base_dir, "Spectrograms")
os.makedirs(output_dir, exist_ok=True)

# Loop through all denoised audio files
for file in os.listdir(input_dir):
    if file.endswith(".wav"):
        file_path = os.path.join(input_dir, file)
        print(f"🎨 Generating spectrogram for {file}...")

        # Load audio
        y, sr = librosa.load(file_path, sr=None)
```

```python
        S_dB = librosa.power_to_db(S, ref=np.max)

        # Plot & save
        plt.figure(figsize=(10, 4))
        librosa.display.specshow(S_dB, sr=sr, hop_length=512, x_axis='time', y_axis='mel', cmap='magma')
        plt.colorbar(format='%+2.0f dB')
        plt.title(f"Mel Spectrogram - {file}")
        plt.tight_layout()

        out_path = os.path.join(output_dir, file.replace(".wav", "_spec.png"))
        plt.savefig(out_path)
        plt.close()

        print(f"✅ Saved: {out_path}")

print("🎉 All spectrograms generated successfully!")
```

```
✅ Saved: /content/drive/MyDrive/DDS/Spectrograms/307_Participant_denoised_spec.png
🎨 Generating spectrogram for 302_Participant_denoised.wav...
✅ Saved: /content/drive/MyDrive/DDS/Spectrograms/302_Participant_denoised_spec.png
🎨 Generating spectrogram for 306_Participant_denoised.wav...
✅ Saved: /content/drive/MyDrive/DDS/Spectrograms/306_Participant_denoised_spec.png
🎨 Generating spectrogram for 309_Participant_denoised.wav...
✅ Saved: /content/drive/MyDrive/DDS/Spectrograms/309_Participant_denoised_spec.png
🎨 Generating spectrogram for 304_Participant_denoised.wav...
```

```python
import os
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np
import gc  # garbage collector

# Define directories
base_dir = "/content/drive/MyDrive/DDS"
input_dir = os.path.join(base_dir, "Denoised_Audio")
output_dir = os.path.join(base_dir, "Spectrograms_Optimized")
os.makedirs(output_dir, exist_ok=True)

# Process in small batches
files = [f for f in os.listdir(input_dir) if f.endswith(".wav")]
batch_size = 10  # you can reduce to 5 if still crashing

for i in range(0, len(files), batch_size):
    batch = files[i:i + batch_size]
    print(f"🔹 Processing batch {i//batch_size + 1}/{len(files)//batch_size + 1}")
```

```python
    for file in batch:
        try:
            file_path = os.path.join(input_dir, file)
            y, sr = librosa.load(file_path, sr=None)

            # Compute Mel spectrogram (lighter version)
            S = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=64)
            S_dB = librosa.power_to_db(S, ref=np.max)

            plt.figure(figsize=(8, 3))
            librosa.display.specshow(S_dB, sr=sr, x_axis='time', y_axis='mel', cmap='magma')
            plt.axis('off')  # remove axes for cleaner visuals
            out_path = os.path.join(output_dir, file.replace(".wav", "_spec.png"))
            plt.savefig(out_path, bbox_inches='tight', pad_inches=0)
            plt.close('all')

            print(f"✅ Saved {file}")
        except Exception as e:
            print(f"⚠️ Skipped {file}: {e}")

    # Clear memory between batches
    gc.collect()

print("🎉 All spectrograms generated with memory optimization!")
```

```
🔹 Processing batch 18/19
✅ Saved 417_Participant_denoised.wav
✅ Saved 449_Participant_denoised.wav
✅ Saved 404_Participant_denoised.wav
✅ Saved 446_Participant_denoised.wav
✅ Saved 432_Participant_denoised.wav
✅ Saved 425_Participant_denoised.wav
✅ Saved 316_Participant_denoised.wav
✅ Saved 317_Participant_denoised.wav
✅ Saved 436_Participant_denoised.wav
✅ Saved 419_Participant_denoised.wav
🔹 Processing batch 19/19
✅ Saved 424_Participant_denoised.wav
✅ Saved 430_Participant_denoised.wav
✅ Saved 440_Participant_denoised.wav
✅ Saved 431_Participant_denoised.wav
✅ Saved 442_Participant_denoised.wav
✅ Saved 321_Participant_denoised.wav
✅ Saved 320_Participant_denoised.wav
✅ Saved 427_Participant_denoised.wav
✅ Saved 433_Participant_denoised.wav
🎉 All spectrograms generated with memory optimization!
```

ViT model

```
# ==============================
# Vision Transformer fine-tuning on DAIC-WOZ Spectrograms (PHQ8_Binary)
# - Full fine-tuning (backbone + head)
# - Mixed precision training
# - Class-weighted loss
# - Save best model & embeddings
# ==============================

# 0️⃣ Install required libraries
!pip install -q timm

# 1️⃣ Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# 2️⃣ Imports & basic setup
import os, re, time, copy
import numpy as np
import pandas as pd
from glob import glob
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
```

```python
import timm
from torch.cuda.amp import autocast, GradScaler

# ========== USER CONFIG ==========
BASE_DIR = "/content/drive/MyDrive/DDS"
SPECTROGRAM_DIR = os.path.join(BASE_DIR, "Spectrograms_Optimized")
CSV_DIR = os.path.join(BASE_DIR, "csvs")
RESULTS_DIR = os.path.join(BASE_DIR, "ViT_Results")
os.makedirs(RESULTS_DIR, exist_ok=True)

IMG_SIZE = 224
BATCH_SIZE = 8
EPOCHS = 25
LR = 1e-4
NUM_WORKERS = 2
PATIENCE = 6
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
MODEL_NAME = "vit_base_patch16_224"
EMBEDDING_FILE = os.path.join(RESULTS_DIR, "embeddings.npy")
EMBEDDING_CSV = os.path.join(RESULTS_DIR, "embeddings_index.csv")
BEST_MODEL_PATH = os.path.join(RESULTS_DIR, "best_vit.pth")
SEED = 42
# ================================

torch.manual_seed(SEED)
np.random.seed(SEED)

print("Device:", DEVICE)
```

```python
print("Spectrogram dir:", SPECTROGRAM_DIR)
print("CSV dir:", CSV_DIR)

# 3️⃣ Load & merge label CSVs
csv_files = [
    os.path.join(CSV_DIR, "train_split_Depression_AVEC2017.csv.xls"),
    os.path.join(CSV_DIR, "dev_split_Depression_AVEC2017.csv.xls"),
    os.path.join(CSV_DIR, "full_test_split.csv.xls"),
]

dfs = []
for path in csv_files:
    try:
        df = pd.read_csv(path)
    except Exception:
        df = pd.read_excel(path)
    dfs.append(df)

labels_df = pd.concat(dfs, ignore_index=True)
labels_df = labels_df[["Participant_ID", "PHQ8_Binary"]].drop_duplicates().reset_index(drop=True)
labels_df["Participant_ID"] = labels_df["Participant_ID"].astype(str)
print("Combined labels:", labels_df.shape)
print(labels_df.head())

# 4️⃣ Match spectrograms with labels
def extract_id(fname):
    m = re.match(r"(\d+)", fname)
    return m.group(1) if m else None
```

```python
spec_files = sorted([f for f in os.listdir(SPECTROGRAM_DIR) if f.lower().endswith(".png")])
rows = [{"filename": f, "Participant_ID": extract_id(f)} for f in spec_files]
files_df = pd.DataFrame(rows)

merged = files_df.merge(labels_df, on="Participant_ID", how="left")
merged = merged.dropna(subset=["PHQ8_Binary"])
merged["label"] = merged["PHQ8_Binary"].astype(int)
print("Labeled spectrograms:", len(merged))
print(merged.head())

# 5️⃣ Train/validation split
train_df, val_df = train_test_split(
    merged, test_size=0.2, stratify=merged["label"], random_state=SEED
)
print("Train/Val sizes:", len(train_df), len(val_df))

# 6️⃣ Dataset & transforms
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]

train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomApply([transforms.ColorJitter(0.1,0.1,0.1,0.1)], p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=mean, std=std),
])
```

```python
val_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=mean, std=std),
])

class SpectrogramDataset(Dataset):
    def __init__(self, df, root_dir, transform=None):
        self.df = df.reset_index(drop=True)
        self.root = root_dir
        self.transform = transform
    def __len__(self):
        return len(self.df)
    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(os.path.join(self.root, row["filename"])).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, int(row["label"]), row["filename"]

train_loader = DataLoader(SpectrogramDataset(train_df, SPECTROGRAM_DIR, train_transform),
                          batch_size=BATCH_SIZE, shuffle=True, num_workers=NUM_WORKERS)
val_loader = DataLoader(SpectrogramDataset(val_df, SPECTROGRAM_DIR, val_transform),
                        batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS)
```

```python
# 7 Build ViT model
model = timm.create_model(MODEL_NAME, pretrained=True, num_classes=0)
in_ch = model.num_features
model.head = nn.Sequential(
    nn.Linear(in_ch, 256),
    nn.GELU(),
    nn.Dropout(0.3),
    nn.Linear(256, 2)
)
model.to(DEVICE)

# 8 Loss, optimizer, scheduler
labels_all = merged["label"].values
class_counts = np.bincount(labels_all)
weights = torch.tensor([(len(labels_all)/(2*c)) if c>0 else 1.0 for c in class_counts], dtype=torch.float32)
print("Class weights:", weights)

criterion = nn.CrossEntropyLoss(weight=weights.to(DEVICE))
optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=0.5, patience=3)

# 9 Training loop
scaler = GradScaler()
best_acc, best_state, no_improve = 0.0, None, 0
start = time.time()

for epoch in range(1, EPOCHS + 1):
    model.train()
```

```python
    train_loss, preds, labels = 0.0, [], []
    for imgs, lbls, _ in train_loader:
        imgs, lbls = imgs.to(DEVICE), lbls.to(DEVICE)
        optimizer.zero_grad()
        with autocast():
            out = model(imgs)
            loss = criterion(out, lbls)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

        train_loss += loss.item() * imgs.size(0)
        preds.extend(out.argmax(1).cpu().tolist())
        labels.extend(lbls.cpu().tolist())

    train_loss /= len(train_loader.dataset)
    train_acc = accuracy_score(labels, preds)

    # --- Validation ---
    model.eval()
    val_loss, vpreds, vlabels = 0.0, [], []
    with torch.no_grad():
        for imgs, lbls, _ in val_loader:
            imgs, lbls = imgs.to(DEVICE), lbls.to(DEVICE)
```

```
            imgs, lbls = imgs.to(DEVICE), lbls.to(DEVICE)
            with autocast():
                out = model(imgs)
                loss = criterion(out, lbls)
            val_loss += loss.item() * imgs.size(0)
            vpreds.extend(out.argmax(1).cpu().tolist())
            vlabels.extend(lbls.cpu().tolist())

    val_loss /= len(val_loader.dataset)
    val_acc = accuracy_score(vlabels, vpreds)
    scheduler.step(val_loss)

    print(f"Epoch {epoch:02d}: Train {train_acc:.3f} | Val {val_acc:.3f} | Loss {val_loss:.4f}")

    if val_acc > best_acc:
        best_acc, best_state = val_acc, copy.deepcopy(model.state_dict())
        torch.save(best_state, BEST_MODEL_PATH)
        print(f"✅ Best model saved (Val Acc: {best_acc:.3f})")
        no_improve = 0
    else:
        no_improve += 1
        if no_improve >= PATIENCE:
            print("⏹ Early stopping triggered.")
            break

print(f"Training done in {(time.time()-start)/60:.1f} min. Best Val Acc={best_acc:.3f}")

if best_state:
```

```
if best_state:
    model.load_state_dict(best_state)

# 🔟 Extract embeddings for all spectrograms
model.eval()
embeddings, meta = [], []
embed_loader = DataLoader(SpectrogramDataset(merged, SPECTROGRAM_DIR, val_transform),
                          batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS)
with torch.no_grad():
    for imgs, lbls, fnames in embed_loader:
        imgs = imgs.to(DEVICE)
        feats = model.forward_features(imgs)
        if isinstance(feats, tuple):
            feats = feats[0]
        embeddings.append(feats.cpu().numpy())
        meta += [{"filename": f, "label": int(l)} for f, l in zip(fnames, lbls)]

emb_array = np.concatenate(embeddings)
np.save(EMBEDDING_FILE, emb_array)
pd.DataFrame(meta).to_csv(EMBEDDING_CSV, index=False)
print(f"✅ Embeddings saved at {EMBEDDING_FILE} (shape {emb_array.shape})")

# 🔁 Final evaluation
all_loader = DataLoader(SpectrogramDataset(merged, SPECTROGRAM_DIR, val_transform),
                        batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS)
y_true, y_pred = [], []
with torch.no_grad():
    for imgs, lbls, _ in all_loader:
```

```
        outs = model(imgs.to(DEVICE))
        y_pred.extend(outs.argmax(1).cpu().tolist())
        y_true.extend(lbls.tolist())

print("🎯 Final overall accuracy:", accuracy_score(y_true, y_pred))
print("✅ All results stored in:", RESULTS_DIR)
```

```
  with autocast():
/usr/local/lib/python3.12/dist-packages/torch/amp/autocast_mode.py:266: UserWarning: User provided device_type of 'cuda', but CUDA is not available. Disabling
  warnings.warn(
/tmp/ipython-input-2231913639.py:191: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with autocast():
/usr/local/lib/python3.12/dist-packages/torch/amp/autocast_mode.py:266: UserWarning: User provided device_type of 'cuda', but CUDA is not available. Disabling
  warnings.warn(
Epoch 02: Train 0.593 | Val 0.310 | Loss 0.6934
/tmp/ipython-input-2231913639.py:171: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with autocast():
/usr/local/lib/python3.12/dist-packages/torch/amp/autocast_mode.py:266: UserWarning: User provided device_type of 'cuda', but CUDA is not available. Disabling
  warnings.warn(
/tmp/ipython-input-2231913639.py:191: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with autocast():
/usr/local/lib/python3.12/dist-packages/torch/amp/autocast_mode.py:266: UserWarning: User provided device_type of 'cuda', but CUDA is not available. Disabling
  warnings.warn(
Epoch 03: Train 0.566 | Val 0.310 | Loss 0.7397
/tmp/ipython-input-2231913639.py:171: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
Epoch 07: Train 0.646 | Val 0.655 | Loss 0.6828
⏹ Early stopping triggered.
Training done in 27.0 min. Best Val Acc=0.690
✅ Embeddings saved at /content/drive/MyDrive/DDS/ViT_Results/embeddings.npy (shape (142, 197, 768))
🎯 Final overall accuracy: 0.704225352112676
✅ All results stored in: /content/drive/MyDrive/DDS/ViT_Results
```

```
# ==========================================
# 📊 1️⃣ Visualize performance + embeddings check
# ==========================================
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc
import seaborn as sns
import torch

SAVE_DIR = "/content/drive/MyDrive/DDS/ViT_Results_Finetuned"

# Load new embeddings
embeddings = np.load(f"{SAVE_DIR}/embeddings_finetuned.npy")
labels = np.load(f"{SAVE_DIR}/labels_finetuned.npy")

print("Embeddings:", embeddings.shape, "Labels:", labels.shape)

# ==========================================
# 2️⃣ Train a lightweight classifier on embeddings
# ==========================================
# Flatten ViT features if they have >2 dims
if len(embeddings.shape) > 2:
    embeddings = embeddings.reshape(embeddings.shape[0], -1)

# Split train/val
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(
    embeddings, labels, test_size=0.2, stratify=labels, random_state=42
```

```python
)

clf = LogisticRegression(max_iter=500, class_weight="balanced")
clf.fit(X_train, y_train)
y_pred = clf.predict(X_val)
y_prob = clf.predict_proba(X_val)[:, 1]

# Metrics
cm = confusion_matrix(y_val, y_pred)
acc = (y_pred == y_val).mean()
fpr, tpr, _ = roc_curve(y_val, y_prob)
roc_auc = auc(fpr, tpr)

print(f"✅ Logistic Regression Accuracy: {acc:.3f}")
print(classification_report(y_val, y_pred, digits=3))

# Confusion Matrix
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix (Fine-Tuned Embeddings)")
plt.show()

# ROC Curve
plt.figure(figsize=(5,4))
plt.plot(fpr, tpr, label=f"AUC={roc_auc:.3f}")
plt.plot([0,1],[0,1],'--',color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

```python
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Fine-Tuned Model")
plt.legend()
plt.show()

print("🎯 Visualization + classifier evaluation complete!")
```

```
Embeddings: (142, 197, 768) Labels: (142,)
✅ Logistic Regression Accuracy: 0.621
              precision    recall  f1-score   support

           0      0.680     0.850     0.756        20
           1      0.250     0.111     0.154         9

    accuracy                          0.621        29
   macro avg      0.465     0.481     0.455        29
weighted avg      0.547     0.621     0.569        29
```

References

1. Zhang X, Li C, Chen W, Zheng J, Li F. Optimizing depression detection in clinical doctor-patient interviews using a multi-instance learning framework. Sci Rep. 2025 Feb 24;15(1):6637. doi: 10.1038/s41598-025-90117-w. PMID: 39994325; PMCID: PMC11850819. https://pmc.ncbi.nlm.nih.gov/articles/PMC11850819/
2. Milintsevich K, Sirts K, Dias G. Towards automatic text-based estimation of depression through symptom prediction. Brain Inform. 2023 Feb 13;10(1):4. doi: 10.1186/s40708-023-00185-9. PMID: 36780049; PMCID: PMC9925661. https://link.springer.com/article/10.1186/s40708-023-00185-9
3. Burdisso, S., Reyes-Ramírez, E., Villatoro-tello, E., Sánchez-Vega, F., Lopez Monroy, A., & Motlicek, P. (2024, June). DAIC-WOZ: On the Validity of Using the Therapist's prompts in Automatic Depression Detection from Clinical Interviews. In T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, & D. Bitterman (Eds), Proceedings of the 6th Clinical Natural Language Processing Workshop (pp. 82–90). https://aclanthology.org/2024.clinicalnlp-1.8
4. Teferra BG, Perivolaris A, Hsiang WN, Sidharta CK, Rueda A, et al. (2025) Leveraging large language models for automated depression screening. PLOS Digital Health 4(7): e0000943 https://journals.plos.org/digitalhealth/article?id=10.1371/journal.pdig.0000943
5. Tank, C., Pol, S., Katoch, V., Mehta, S., Anand, A., & Shah, R. R. (2024). Depression Detection and Analysis using Large Language Models on Textual and Audio-Visual Modalities. arXiv [Cs.HC]. https://arxiv.org/abs/2407.06125
6. Triohin V, Leba M, Ionica AC. From Convolution to Spikes for Mental Health: A CNN-to-SNN Approach Using the DAIC-WOZ Dataset. Applied Sciences. 2025; 15(16):9032. https://www.mdpi.com/2076-3417/15/16/9032
7. Lin L, Chen X, Shen Y, Zhang L. Towards Automatic Depression Detection: A BiLSTM/1D CNN-Based Model. Applied Sciences. 2020; 10(23):8701. https://doi.org/10.3390/app10238701
8. Z. Zhou, Y. Guo, S. Hao and R. Hong, "Hierarchical Multifeature Fusion via Audio-Response-Level Modeling for Depression Detection," in IEEE Transactions on Computational Social Systems, vol. 10, no. 5, pp. 2797-2805, Oct. 2023, doi: 10.1109/TCSS.2022.3202294. https://ieeexplore.ieee.org/abstract/document/9885023

9. Ravi V, Wang J, Flint J, Alwan A. A Step Towards Preserving Speakers' Identity While Detecting Depression Via Speaker Disentanglement. Interspeech. 2022 Sep;2022:3338-3342. doi:10.21437/interspeech.2022-10798. PMID: 36341467; PMCID: PMC9635494. https://pmc.ncbi.nlm.nih.gov/articles/PMC9635494/
10. Feng K, Chaspari T. A knowledge-driven vowel-based approach of depression classification from speech using data augmentation. ICASSP 2023—2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE; 2023:1-5.
11. Nykoniuk M, Basystiuk O, Shakhovska N, Melnykova N. Multimodal Data Fusion for Depression Detection Approach. Computation. 2025; 13(1):9. https://doi.org/10.3390/computation13010009
12. Iyortsuun, Ngumimi & Kim, S.H. & Yang, Hyung-Jeong & Kim, Seung-Won & Jhon, Min. (2024). Additive Cross-Modal Attention Network (ACMA) for Depression Detection Based on Audio and Textual Features. IEEE Access. PP. 1-1. 10.1109/ACCESS.2024.3362233. https://www.researchgate.net/publication/377996131_Additive_Cross-Modal_Attention_Network_ACMA_for_Depression_Detection_based_on_Audio_and_Textual_Features
13. Jo, A-Hyeon & Kwak, Keun-Chang. (2022). Diagnosis of Depression Based on Four-Stream Model of Bi-LSTM and CNN from Audio and Text Information. IEEE Access. PP. 1-1. 10.1109/ACCESS.2022.3231884. https://www.researchgate.net/publication/366566198_Diagnosis_of_Depression_Based_on_Four-Stream_Model_of_Bi-LSTM_and_CNN_from_Audio_and_Text_Information
14. Afzal Aghaei, Alireza & Khodaei, Nadia. (2024). Automated Depression Recognition Using Multimodal Machine Learning: A Study on the DAIC-WOZ Dataset. Computational Mathematics and Computer Modelling with Applications (CMCMA). 2. 45. 10.48308/CMCMA.2.1.45. https://www.researchgate.net/publication/379757572_Automated_Depression_Recognition_Using_Multimodal_Machine_Learning_A_Study_on_the_DAIC-WOZ_Dataset
15. Shen, Y., Yang, H., & Lin, L. (2022). Automatic Depression Detection: An Emotional Audio-Textual Corpus and a GRU/BiLSTM-based Model. arXiv [Eess.AS]. http://arxiv.org/abs/2202.08210